

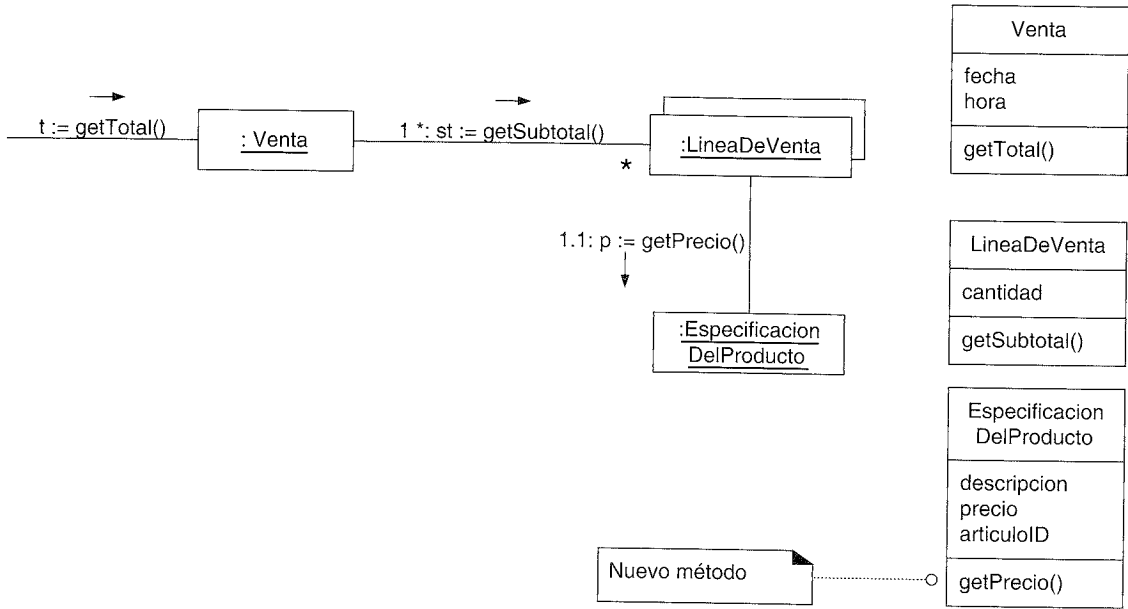


Figura 16.6. Cálculo del total de una Venta.

que la información se encuentre esparcida por objetos diferentes, necesitarán interactuar mediante el paso de mensajes para compartir el trabajo.

Por lo general, el Experto nos conduce a diseños donde los objetos del software realizan aquellas operaciones que normalmente se hacen a los objetos inanimados del mundo real que representan; Peter Coad denomina a esto la estrategia “Hacerlo yo mismo” [Coad95]. Por ejemplo, en el mundo real, sin el uso de ayudas mecánicas, una venta no te dice su total; es un objeto inanimado. Alguien calcula el total de la venta. Pero en el campo del software orientado a objetos, todos los objetos software están “vivos” o “animados”, y pueden tener responsabilidades y hacer cosas. Fundamentalmente, hacen cosas relacionadas con la información que conocen. Yo lo denomino el principio “de animación” en el diseño de objetos; es como estar en unos dibujos animados donde todo está vivo.

El patrón Experto en Información —como muchas cosas en la tecnología de objetos— tiene una analogía en el mundo real. Normalmente, otorgamos responsabilidades a los individuos con la información necesaria para llevar a cabo una tarea. Por ejemplo, en un negocio, ¿quién debería ser el responsable de la creación de una declaración de ganancias y pérdidas? La persona que tiene acceso a toda la información necesaria para crearla —quizás el director financiero—. Y, del mismo modo en que los objetos colaboran porque la información se encuentra dispersa, así pasa con las personas. El director financiero de la compañía podría solicitar a los contables que generen informes sobre los créditos y débitos.

En algunas ocasiones la solución que sugiere el Experto no es deseable, normalmente debido a problemas de acoplamiento y cohesión (estos principios se discutirán más tarde en este capítulo).

Por ejemplo, ¿quién debería ser el responsable de almacenar una *Venta* en una base de datos? Ciertamente, mucha de la información que se tiene que almacenar se encuen-

tra en el objeto *Venta* y, por tanto, siguiendo el patrón Experto se podría argumentar la inclusión de la responsabilidad en la clase *Venta*. La extensión lógica de esta decisión es que cada clase contiene sus propios servicios para almacenarse ella misma en una base de datos. Pero esto nos lleva a problemas de cohesión, acoplamiento y duplicación. Por ejemplo, la clase *Venta* debe ahora contener la lógica relacionada con la gestión de la base de datos, como la relacionada con SQL y JDBC (Conexión de Base de Datos de Java, *Java Database Connectivity*). La clase ya no está centrada únicamente en la lógica de la aplicación de “ser una venta” simplemente; ahora tiene otro tipo de responsabilidades, lo que disminuye su cohesión. La clase debe acoplarse a los servicios técnicos de base de datos de otro subsistema, como los servicios JDBC, en lugar de acoplarse únicamente con otros objetos en la capa del dominio de los objetos del software, lo que eleva su acoplamiento. Y es probable que se dupliquen lógicas de bases de datos similares en muchas clases persistentes.

Todos estos problemas indican la violación de un principio arquitectural básico: diseñe separando los principales aspectos del sistema. Mantenga la lógica de la aplicación en un sitio (como en los objetos software del dominio), mantenga la lógica de la base de datos en otro sitio (como en un subsistema de servicios de persistencia separado), y así sucesivamente, en lugar de entremezclar aspectos del sistema diferentes en el mismo componente⁴.

Separando los aspectos importantes se mejora el acoplamiento y la cohesión del diseño. Por tanto, aunque siguiendo el Experto se podría justificar la asignación de la responsabilidad para el servicio de la base de datos a la clase *Venta*, por otros motivos (normalmente cohesión y acoplamiento), resulta un diseño pobre.

Beneficios

- Se mantiene el encapsulamiento de la información, puesto que los objetos utilizan su propia información para llevar a cabo las tareas. Normalmente, esto conlleva un bajo acoplamiento, lo que da lugar a sistemas más robustos y más fáciles de mantener. (Bajo Acoplamiento también es un patrón GRASP que se estudiará en una de las secciones siguientes.)
- Se distribuye el comportamiento entre las clases que contienen la información requerida, por tanto, se estimula las definiciones de clases más cohesivas y “ligeras” que son más fáciles de entender y mantener. Se soporta normalmente una alta cohesión (otro patrón que se estudiará más tarde).
- Bajo Acoplamiento.
- Alta Cohesión.

Patrones o Principios Relacionados

También conocido como; Parecido a

“Colocar las responsabilidades con los datos”, “Eso que conoces, hazlo”, “Hacerlo yo mismo”, “Colocar los Servicios con los Atributos con los que trabaja”.

16.7. Creador

Solución

Asignar a la clase B la responsabilidad de crear una instancia de clase A si se cumple uno o más de los casos siguientes:

⁴ En el Capítulo 32 se presentará una discusión sobre la separación de intereses.

Contraindicaciones

- B *agrega* objetos de A.
- B *contiene* objetos de A.
- B *registra* instancias de objetos de A.
- B *utiliza más estrechamente* objetos de A.
- B *tiene los datos de inicialización* que se pasarán a un objeto de A cuando sea creado (por tanto, B es un Experto con respecto a la creación de A).

B es un *creador* de los objetos A.

Si se puede aplicar más de una opción, inclínese por una clase B que *agregue* o *contenga* la clase A.

Problema ¿Quién debería ser el responsable de la creación de una nueva instancia de alguna clase?

La creación de instancias es una de las actividades más comunes en un sistema orientado a objetos. En consecuencia, es útil contar con un principio general para la asignación de las responsabilidades de creación. Si se asignan bien, el diseño puede soportar un bajo acoplamiento, mayor claridad, encapsulación y reutilización.

Ejemplo En la aplicación del PDV, ¿quién debería ser el responsable de la creación de una instancia de *LineaDeVenta*? Según el patrón Creador, deberíamos buscar las clases que agregan, contienen, etcétera, instancias de *LineaDeVenta*. Considere el modelo del dominio parcial de la Figura 16.7.

Puesto que una *Venta* contiene (de hecho, agrega) muchos objetos de *LineaDeVenta*, el patrón Creador sugiere que *Venta* es un buen candidato para tener la responsabilidad de la creación de las instancias de *LineaDeVenta*.

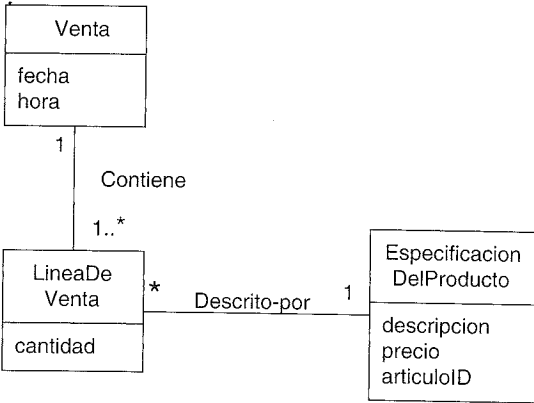


Figura 16.7. Modelo del Dominio Parcial.

Esto nos lleva al diseño de las interacciones entre los objetos que se muestran en la Figura 16.8.

Esta asignación de responsabilidades requiere que se defina en la clase *Venta* un método *crearLineaDeVenta*.

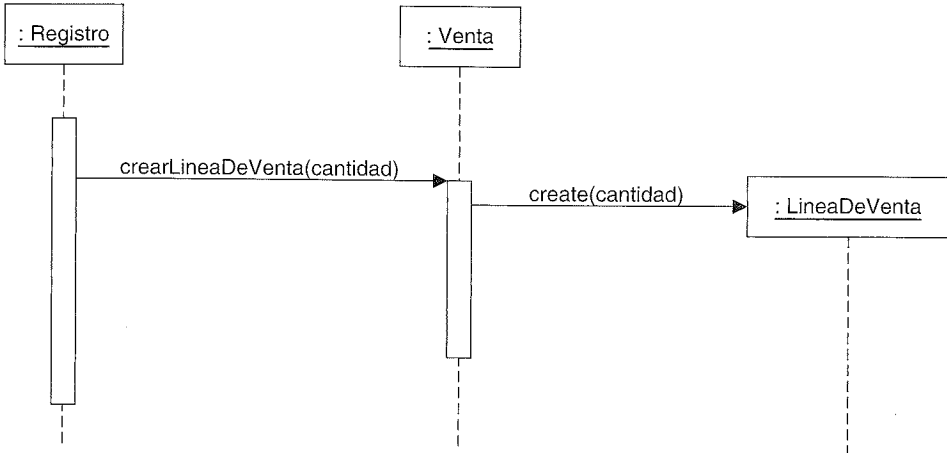


Figura 16.8. Creación de una LineaDeVenta.

Discusión

Una vez más, el contexto en el que se consideraron y se optaron por estas responsabilidades fue durante la elaboración de un diagrama de interacción. La sección de los métodos de un diagrama de clases puede mostrar los resultados de la asignación de responsabilidades, materializadas concretamente como métodos.

El patrón Creador guía la asignación de responsabilidades relacionadas con la creación de objetos, una tarea muy común. La intención básica del patrón Creador es encontrar un creador que necesite conectarse al objeto creado en alguna situación. Elijiéndolo como el creador se favorece el bajo acoplamiento.

El Agregado *agrega* Partes, el Contenedor *contiene* Contenido, y el Registro *registra* Registros, son todas ellas relaciones comunes entre las clases en un diagrama de clases. El Creador sugiere que la clase contenedor o registro es una buena candidata para asignarle la responsabilidad de crear lo que contiene o registra. Por su puesto, esto es sólo una guía.

Nótese que se ha utilizado el concepto de **agregación** al considerar el patrón Creador. La agregación se examinará en el Capítulo 27; una definición breve es que una agregación involucra cosas que se encuentran en una relación Todo-Parte o Ensamblaje-Parte, como un Cuerpo agrega Pierna o un Párrafo agrega Frase.

Algunas veces se encuentra un creador buscando las clases que tienen los datos de inicialización que se pasarán durante la creación. Se trata, en realidad, de un ejemplo del patrón Experto. Los datos de inicialización se pasan durante la creación por medio de algún tipo de método de inicialización, como un constructor Java que tiene parámetros. Por ejemplo, asuma que una instancia de *Pago* necesita inicializarse, cuando se crea, con el total de la *Venta*. Puesto que la *Venta* conoce el total, la *Venta* es un creador candidato del *Pago*.

Contraindicaciones

A menudo, la creación requiere una complejidad significativa, como utilizar instancias recicladas por motivos de rendimiento, crear condicionalmente una instancia a partir de una familia de clases similares basado en el valor de alguna propiedad externa, etcétera. En estos casos, es aconsejable delegar la creación a una clase auxiliar denominada *Factoría* [GHJV95] en lugar de utilizar la clase que sugiere el *Creador*. Las factorías se estudiarán en el Capítulo 23.

- Beneficios**
- Se soporta bajo acoplamiento (descrito a continuación), lo que implica menos dependencias de mantenimiento y mayores oportunidades para reutilizar. Probablemente no se incrementa el acoplamiento porque la clase *creada* es presumible que ya sea visible a la clase *creadora*, debido a las asociaciones existentes que motivaron su elección como creador.
- Patrones o Principios Relacionados**
- Bajo Acoplamiento.
 - Factoría.
 - Todo-Parte [BMRSS96] describe un patrón para definir objetos agregados que favorece la encapsulación de componentes.

16.8. Bajo Acoplamiento

Solución

Asignar una responsabilidad de manera que el acoplamiento permanezca bajo.

Problema

¿Cómo soportar bajas dependencias, bajo impacto del cambio e incremento de la reutilización?

El **acoplamiento** es una medida de la fuerza con que un elemento está conectado a, tiene conocimiento de, confía en, otros elementos. Un elemento con bajo (o débil) acoplamiento no depende de demasiados otros elementos; “demasiados” depende del contexto, pero se estudiará. Estos elementos pueden ser clases, subsistemas, sistemas, etcétera.

Una clase con alto (o fuerte) acoplamiento confía en muchas otras clases. Tales clases podrían no ser deseables; algunas adolecen de los siguientes problemas:

- Los cambios en las clases relacionadas fuerzan cambios locales.
- Son difíciles de entender de manera aislada.
- Son difíciles de reutilizar puesto que su uso requiere la presencia adicional de las clases de las que depende.

Ejemplo

Considere el siguiente diagrama de clases parcial del caso de estudio NuevaEra:

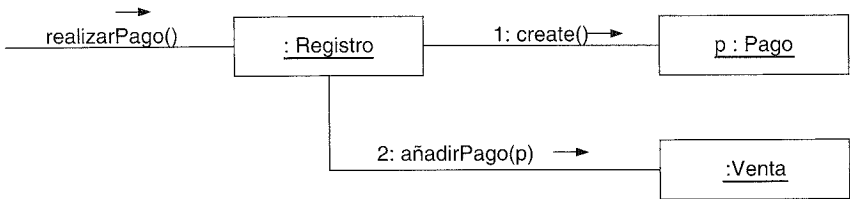
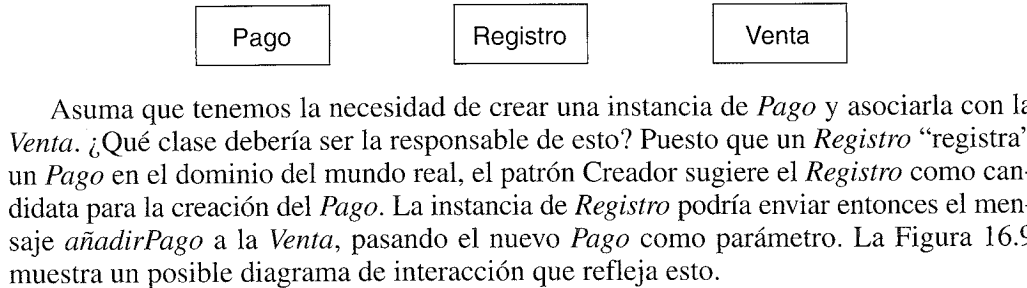


Figura 16.9. El Registro crea un Pago.

Esta asignación de responsabilidades acopla la clase *Registro* con el conocimiento de la clase *Pago*.

Notación UML: Nótese que la instancia de *Pago* se nombra explícitamente con una *p* de manera que se puede referenciar como parámetro en el mensaje 2.

La Figura 16.10 muestra una solución alternativa a crear el *Pago* y asociarlo con la *Venta*.

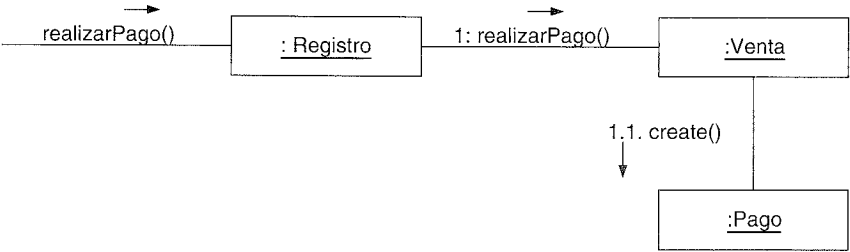


Figura 16.10. La Venta crea el Pago.

¿Qué diseño, basado en la asignación de responsabilidades, soporta Bajo Acoplamiento? En ambos casos asumiremos que la *Venta* debe finalmente acoplarse con el conocimiento del *Pago*. El Diseño 1, en el que el *Registro* crea el *Pago*, añade acoplamiento entre el *Registro* y el *Pago*, mientras que el Diseño 2, en el que la *Venta* lleva a cabo la creación del *Pago*, no incrementa el acoplamiento. Desde el punto de vista puramente del acoplamiento, es preferible el Diseño 2 porque mantiene el acoplamiento global más bajo. Éste es un ejemplo en el que dos patrones —Bajo Acoplamiento y Creador— podrían sugerir soluciones diferentes.

En la práctica, el nivel de acoplamiento no se puede considerar de manera aislada a otros principios como el Experto o Alta Cohesión. Sin embargo, es un factor a tener en cuenta para mejorar un diseño.

Discusión

El patrón de Bajo Acoplamiento es un principio a tener en mente en todas las decisiones de diseño; es un objetivo subyacente a tener en cuenta continuamente. Es un **principio evaluativo** que aplica un diseñador mientras evalúa todas las decisiones de diseño.

En los lenguajes orientados a objetos como C++, Java y C#, algunas de las formas comunes de acoplamiento entre el *TipoX* y el *TipoY* son:

- El *TipoX* tiene un atributo (miembro de datos, o variable de instancia) que hace referencia a una instancia de *TipoY*, o al propio *TipoY*.
- Un objeto de *TipoX* invoca los servicios de un objeto de *TipoY*.
- El *TipoX* tiene un método que referencia a una instancia de *TipoY*, o al propio *TipoY*, de algún modo. Esto, generalmente, comprende un parámetro o variable local de *TipoY*, o que el objeto de retorno de un mensaje sea una instancia de *TipoY*.
- El *TipoX* es una subclase, directa o indirecta, del *TipoY*.
- El *TipoY* es una interfaz y el *TipoX* implementa esa interfaz.

El patrón Bajo Acoplamiento impulsa la asignación de responsabilidades de manera que su localización no incremente el acoplamiento hasta un nivel que nos lleve a los resultados negativos que puede producir un acoplamiento alto.

El Bajo Acoplamiento soporta el diseño de clases que son más independientes, lo que reduce el impacto del cambio. No se puede considerar de manera aislada a otros patrones como el Experto o el de Alta Cohesión, sino que necesita incluirse como uno de los diferentes principios de diseño que influyen en una elección al asignar una responsabilidad.

Una subclase está fuertemente acoplada a su superclase. Se debe estudiar cuidadosamente la decisión de derivar a partir de una superclase debido a esta forma fuerte de acoplamiento. Por ejemplo, suponga que necesitamos almacenar los objetos de manera persistente en una base de datos relacional u objetual. En este caso, un diseño relativamente común consiste en crear una superclase abstracta denominada *ObjetoPersistente* a partir de la cual derivan otras clases. El inconveniente de esta creación de subclases es que acopla altamente los objetos del dominio a un servicio del nivel tecnológico particular y mezcla diferentes aspectos de la arquitectura, mientras que la ventaja es que se hereda de manera automática el comportamiento persistente.

No existe una medida absoluta de cuando el acoplamiento es demasiado alto. Lo que es importante es que un desarrollador pueda medir el grado de acoplamiento actual, y evaluar si aumentarlo le causará problemas. En general, las clases que son inherentemente muy genéricas por naturaleza, y con una probabilidad de reutilización alta, debería tener un acoplamiento especialmente bajo.

El caso extremo de Bajo Acoplamiento es cuando no existe acoplamiento entre clases. Esto no es deseable porque una metáfora central de la tecnología de objetos es un sistema de objetos conectados que se comunican mediante el paso de mensajes. Si el Bajo Acoplamiento se lleva al extremo, producirá un diseño pobre porque dará lugar a unos pocos objetos inconexos, saturados, y con actividad compleja que hacen todo el trabajo, con muchos objetos muy pasivos, sin acoplamiento que actúan como simples repositorios de datos. Es normal y necesario un grado moderado de acoplamiento entre las clases para crear un sistema orientado a objetos en el que las tareas se llevan a cabo mediante la colaboración de los objetos conectados.

No suele ser un problema el acoplamiento alto entre objetos estables y elementos generalizados. Por ejemplo, una aplicación Java J2EE puede acoplarse con seguridad con las librerías de Java (*java.util*, etc.), porque son estables y extendidas.

Escoja sus batallas

El alto acoplamiento en sí no es un problema; es el alto acoplamiento entre elementos que no son estables en alguna dimensión, como su interfaz, implementación o su mera presencia.

Éste es un punto importante: como diseñadores, podemos añadir flexibilidad, encapsular detalles e implementaciones, y, en general, diseñar para disminuir el acoplamiento en muchas áreas del sistema. Pero, si nos esforzamos en “futuras necesidades” o en disminuir el acoplamiento en algunos puntos donde de hecho no hay motivos realistas, el tiempo no se está empleando de manera adecuada.

Contraindicaciones

Los diseñadores tienen que escoger sus batallas para disminuir el acoplamiento y encapsular información. Centrándose en los puntos en los que sea realista pensar en una inestabilidad o evolución alta. Por ejemplo, en el proyecto NuevaEra, se sabe que se necesitan conectar al sistema diferentes sistemas calculadores de impuestos de terceras partes (con interfaces únicas). Por tanto, es práctico diseñar para disminuir el acoplamiento en este punto de variación.

- Beneficios
- No afectan los cambios en otros componentes.
 - Fácil de entender de manera aislada.
 - Conveniente para reutilizar.

Antecedentes

El acoplamiento y la cohesión (descrita a continuación) son principios realmente fundamentales en el diseño y todos los desarrolladores de software deberían apreciarlos y aplicarlos por sí mismos. Larry Constantine, también un creador del diseño estructurado en los años setenta y que actualmente propugna una mayor atención a la ingeniería de usabilidad [CL99], fue el principal responsable en los sesenta de la identificación y comunicación del acoplamiento y la cohesión como principios básicos [Constantine68, CMS74].

- Patrones Relacionados
- Variaciones Protegidas.

16.9. Alta Cohesión

Solución

Asignar una responsabilidad de manera que la cohesión permanezca alta.

Problema

¿Cómo mantener la complejidad manejable?

En cuanto al diseño de objetos, la **cohesión** (o de manera más específica, la cohesión funcional) es una medida de la fuerza con la que se relacionan y del grado de focalización de las responsabilidades de un elemento. Un elemento con responsabilidades altamente relacionadas, y que no hace una gran cantidad de trabajo, tiene alta cohesión. Estos elementos pueden ser clases, subsistemas, etcétera.

Una clase con baja cohesión hace muchas cosas no relacionadas, o hace demasiado trabajo. Tales clases no son convenientes; adolecen de los siguientes problemas:

- Difíciles de entender.
- Difíciles de reutilizar.
- Difíciles de mantener.
- Delicadas, constantemente afectadas por los cambios.

A menudo, las clases con baja cohesión representan un “grano grande” de abstracción, o se les han asignado responsabilidades que deberían haberse delegado en otros objetos.

Podemos analizar para la Alta Cohesión el mismo ejemplo que se utilizó para el patrón de Bajo Acoplamiento.

Asuma que necesitamos crear una instancia de *Pago* (en efectivo) y asociarla con la *Venta*. ¿Qué clase debería ser responsable de esto? Puesto que el *Registro* registra un *Pago* en el dominio del mundo real, el patrón *Creador* sugiere el *Registro* como candidato para la creación del *Pago*. La instancia de *Registro* podría entonces enviar un

Ejemplo

mensaje *añadirPago* a la *Venta*, pasando como parámetro el nuevo *Pago*, como se muestra en la Figura 16.11.

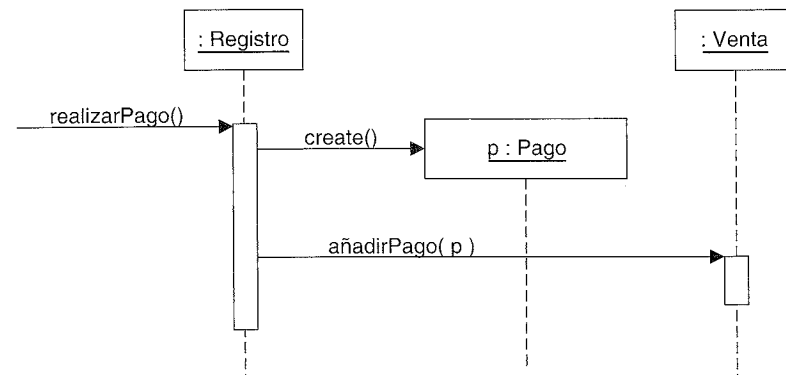


Figura 16.11. El Registro crea el Pago.

Esta asignación de responsabilidades sitúa la responsabilidad de realizar un pago en el *Registro*. El *Registro* toma parte en la responsabilidad de llevar a cabo la operación del sistema *realizarPago*.

En este ejemplo aislado, esto es aceptable; pero si continuamos haciendo responsable a la clase *Registro* de realizar parte o la mayoría del trabajo relacionado con más y más operaciones del sistema, se irá sobrecargando incrementalmente con tareas y llegará a perder la cohesión.

Imagine que hubiera cincuenta operaciones del sistema, todas recibidas por *Registro*. Si hace el trabajo relacionado con cada una, se convertirá en un objeto "saturado" y sin cohesión. El hecho no es que esta simple tarea de creación de un *Pago* en sí misma haga que el *Registro* no sea cohesivo, sino que como parte de un cuadro mayor de asignación de responsabilidades global, podría sugerir una tendencia a una baja cohesión.

Y más importante en cuanto a las habilidades de desarrollo como diseñadores de objetos, independientemente de la elección de diseño final, lo importante es que por lo menos un desarrollador sepa tener en cuenta el impacto de la cohesión.

En cambio, como se muestra en la Figura 16.12, el segundo diseño delega la responsabilidad de creación del pago a la *Venta*, que favorece una cohesión más alta en el *Registro*.

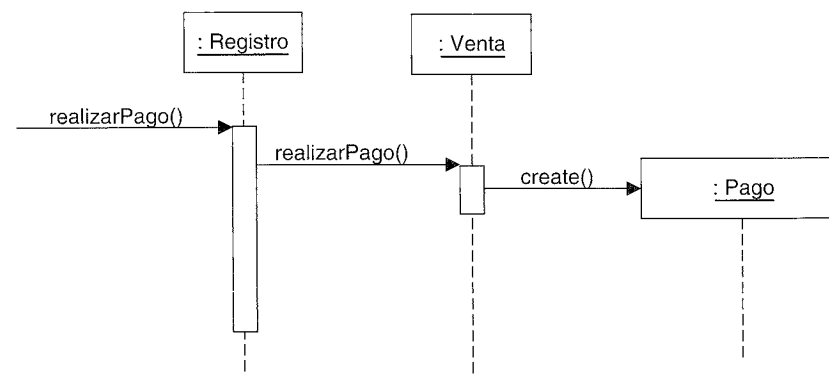


Figura 16.12. La Venta crea el Pago.

Es deseable el segundo diseño puesto que soporta tanto alta cohesión como bajo acoplamiento.

En la práctica, el nivel de cohesión no se puede considerar de manera aislada a otras responsabilidades y otros principios como los patrones Experto y Bajo Acoplamiento.

Discusión

Igual que el Bajo Acoplamiento, el patrón de Alta Cohesión es un principio a tener en mente durante todas las decisiones de diseño; es un objetivo subyacente a tener en cuenta continuamente. Es un principio evaluativo que aplica un diseñador mientras evalúa todas las decisiones de diseño.

Grady Booch establece que existe alta cohesión funcional cuando los elementos de un componente (como una clase) "trabajan todos juntos para proporcionar algún comportamiento bien delimitado" [Booch94].

A continuación presentamos algunos escenarios que ilustran diferentes grados de cohesión funcional:

1. *Muy baja cohesión.* Una única clase es responsable de muchas cosas en áreas funcionales muy diferentes.
 - Asuma que existe una clase denominada *Interfaz-BDR-RPC* que es completamente responsable de la interacción con bases de datos relacionales y la gestión de las llamadas remotas a procedimientos (*Remote Procedure Calls*). Éstas son dos áreas funcionales muy diferentes, y cada una requiere mucho código de soporte. La responsabilidad debería dividirse en una familia de clases relacionadas con el acceso BDR y una familia relacionada con el soporte de RPC.
2. *Baja cohesión.* Una única clase tiene la responsabilidad de una tarea compleja en un área funcional.
 - Asuma que existe una clase denominada *InterfazBDR* que es completamente responsable de interactuar con las bases de datos relacionales. Los métodos de la clase están todos relacionados, pero hay muchos, y una gran cantidad de código de soporte; podría haber cientos o miles de métodos. La clase debería dividirse en una familia de clases ligeras que comparten el trabajo de proporcionar el acceso BDR.
3. *Alta cohesión.* Una clase tiene una responsabilidad moderada en un área funcional y colabora con otras clases para llevar a cabo las tareas.
 - Asuma que existe una clase denominada *InterfazBDR* que es sólo parcialmente responsable de interactuar con las bases de datos relacionales. Interactúa con una docena de otras clases relacionadas con el acceso BDR para recuperar y almacenar objetos.
4. *Moderada cohesión.* Una clase tiene responsabilidades ligeras y únicas en unas pocas áreas diferentes que están lógicamente relacionadas con el concepto de la clase, pero no entre ellas.
 - Asuma que existe una clase denominada *Compañía* que es completamente responsable de (a) conocer a sus empleados y (b) conocer su información fi-

nanciera. Estas dos áreas no están fuertemente relacionadas entre ellas; aunque ambas están lógicamente relacionadas con el concepto de compañía. Además, el número total de métodos públicos es pequeño, al igual que la cantidad de código de soporte.

Como regla empírica, una clase con alta cohesión tiene un número relativamente pequeño de métodos, con funcionalidad altamente relacionada, y no realiza mucho trabajo. Colabora con otros objetos para compartir el esfuerzo si la tarea es extensa.

Una clase con alta cohesión es ventajosa porque es relativamente fácil de mantener, entender y reutilizar. El alto grado de funcionalidad relacionada, combinada con un número pequeño de operaciones, también simplifica el mantenimiento y las mejoras. El grano fino de la funcionalidad altamente relacionada también aumenta el potencial de reutilización.

El patrón de Alta Cohesión —como muchas cosas en la tecnología de objetos— tiene una analogía en el mundo real. Una observación típica es que si una persona tiene demasiadas responsabilidades no relacionadas —especialmente unas que deberían delegarse de manera adecuada en otras— entonces la persona no es efectiva. Se observa en algunos directores que no han aprendido a delegar. Estas personas padecen baja cohesión; están preparados para llegar a ser personas “despegadas”.

Otro principio clásico: diseño modular

El acoplamiento y la cohesión son viejos principios del diseño de software; diseñar con objetos no implica que se ignoren los fundamentos bien establecidos. Otro de estos principios —que está fuertemente relacionado con el acoplamiento y la cohesión— es promover el **diseño modular**. Citando textualmente:

La modularidad es la propiedad de un sistema que se ha descompuesto en un conjunto de módulos cohesivos y débilmente acoplados [Booch94].

Fomentamos el diseño modular creando métodos y clases con alta cohesión. Al nivel básico de objetos, la modularidad se alcanza diseñando cada método con un único y claro objetivo, y agrupando un conjunto de aspectos relacionados en una clase.

Cohesión y acoplamiento; el yin y el yang

Una mala cohesión causa, normalmente, un mal acoplamiento, y viceversa. Llamo a la cohesión y el acoplamiento el *yin y el yang de la ingeniería del software* debido a que influye el uno en el otro. Por ejemplo, considere una clase que implementa un elemento gráfico GUI que representa y pinta dicho elemento gráfico, almacena datos en una base de datos, e invoca el servicio de objetos remotos. No sólo es profundamente no cohesiva, sino que está acoplada a muchos (y con nada en común) elementos.

Existen pocos casos en los que esté justificada la aceptación de baja cohesión.

Un caso es la agrupación de responsabilidades o código en una clase o componente para simplificar el mantenimiento por una persona —aunque queda advertido de que tal agrupación podría también empeorar el mantenimiento—. Pero por ejemplo, suponga

que una aplicación contiene sentencias SQL embebidas que siguiendo otros buenos principios de diseño deberían distribuirse por diez clases, tales como diez clases encargadas de la “conversión objeto-tupla de base de datos”. Ahora bien, es normal que sólo uno o dos expertos en SQL conozcan la mejor manera para definir y mantener este SQL, incluso si hay docenas de programadores orientados a objetos (OO) en el proyecto; pocos programadores OO podrían dominar SQL. Suponga que incluso el experto en SQL no se encuentra cómodo como programador OO. El arquitecto de software podría decidir agrupar todas las sentencias SQL en una clase, *OperacionesBDR*, de manera que sea fácil para el experto en SQL trabajar en SQL en un único lugar.

Otro caso de componentes con baja cohesión lo constituyen los objetos servidores distribuidos. Debido a implicaciones de costes fijos y rendimientos asociados con los objetos remotos y comunicaciones remotas, a veces, es deseable crear menos objetos servidores, de mayor tamaño y menos cohesivos que proporcionen una interfaz para muchas operaciones. Esto también está relacionado con el patrón denominado **Interfaz Remota de Grano Grueso**, en el cual, las operaciones remotas se hacen de grano más grueso con el objeto de realizar o solicitar más trabajo en las llamadas a operaciones remotas, debido a las penalizaciones de rendimiento de las llamadas remotas en la red. Como un simple ejemplo, en vez de un objeto remoto con tres operaciones de grano fino *setNombre*⁵, *setSalario*, y *setFechaAlquiler*, hay una operación remota *setDatos* que recibe un conjunto de datos. Esto da lugar a menos llamadas remotas y mejor rendimiento.

Beneficios

- Se incrementa la claridad y facilita la comprensión del diseño.
- Se simplifican el mantenimiento y las mejoras.
- Se soporta a menudo bajo acoplamiento.
- El grano fino de funcionalidad altamente relacionada incrementa la reutilización porque una clase cohesiva se puede utilizar para un propósito muy específico.

16.10. Controlador

Solución

Asignar la responsabilidad de recibir o manejar un mensaje de evento del sistema a una clase que representa una de las siguientes opciones:

- Representa el sistema global, dispositivo o subsistema (*controlador de fachada*).
- Representa un escenario de caso de uso en el que tiene lugar el evento del sistema, a menudo denominado <NombreDelCasoDeUso>Manejador, <NombreDelCasoDeUso>Coordinador o <NombreDelCasoDeUso>Sesion (*controlador de sesión o de caso de uso*).
 - Utilice la misma clase controlador para todos los eventos del sistema en el mismo escenario de caso de uso.
 - Informalmente, una sesión es una instancia de una conversación con un actor. Las sesiones pueden tener cualquier duración, pero se organizan a menudo en función de los casos de uso (sesiones de casos de uso).

Contraindicaciones

⁵ N. del T.: Para nombrar los métodos de una clase que asignan un valor a una propiedad denominada <X>, utilizaremos la habitual terminología set<X>.

Corolario: Nótese que las clases “ventana”, “applet”, “widget”, “vista” y “documento” no están en esta lista. Tales clases *no* deberían abordar las tareas asociadas con los eventos del sistema, normalmente, reciben estos eventos y los delegan a un controlador.

Problema ¿Quién debe ser el responsable de gestionar un evento de entrada al sistema?

Un **evento del sistema** de entrada es un evento generado por un actor externo. Se asocian con **operaciones del sistema** —operaciones del sistema como respuesta a los eventos del sistema—, tal como se relacionan los mensajes y los métodos.

Por ejemplo, cuando un cajero utiliza un terminal PDV y presiona el botón “Finalizar Venta”, está generando un evento del sistema que indica que la “venta ha terminado”. Igualmente, cuando un escritor utiliza un procesador de texto y presiona el botón “comprobar ortografía”, está generando un evento del sistema que indica que se “ejecute una comprobación de la ortografía”.

Un **Controlador** es un objeto que no pertenece a la interfaz de usuario, responsable de recibir o manejar un evento del sistema. Un Controlador define el método para la operación del sistema.

Ejemplo En la aplicación NuevaEra, hay varias operaciones del sistema, como se ilustra en la Figura 16.13, que muestran al propio sistema como una clase o componente (que es legal en UML).

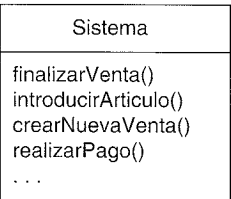


Figura 16.13. Operaciones del sistema asociadas con los eventos del sistema.

Durante el análisis, las operaciones del sistema pueden asignarse a la clase *Sistema*, para indicar que se trata de operaciones del sistema. Sin embargo, esto *no* significa que una clase software denominada *Sistema* las lleve a cabo durante el diseño. Más bien, durante el diseño, se asigna la responsabilidad de las operaciones del sistema a una clase Controlador (ver Figura 16.14).

¿Quién debería ser el controlador de los eventos del sistema como *introducirArticulo* y *finalizarVenta*?

Siguiendo el patrón Controlador, a continuación presentamos algunas de las opciones:

- | | |
|--|---|
| representa el “sistema” global,
dispositivo o subsistema | <i>Registro, SistemaPDV</i> |
| representa un receptor o manejador
de todos los eventos del sistema
de un escenario de caso de uso | <i>ProcesarVentaManejador</i>
<i>ProcesarVentaSesion</i> |

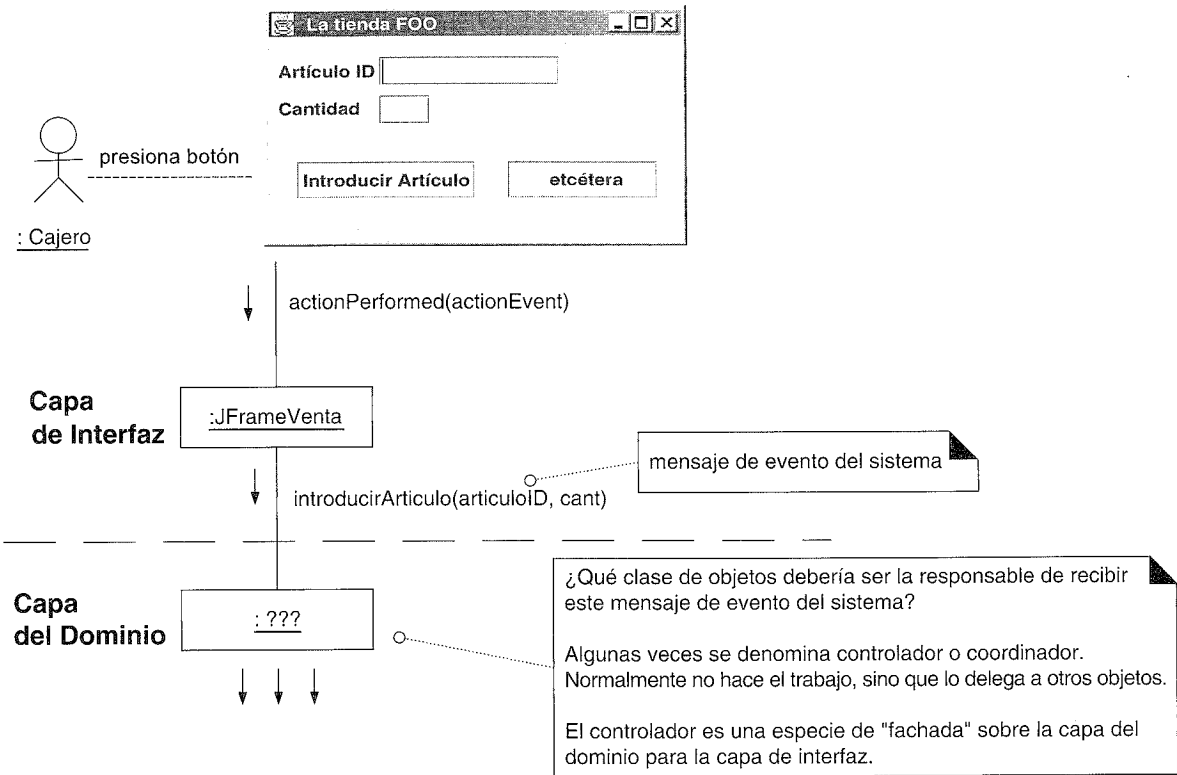


Figura 16.14. ¿Controlador para introducirArticulo?

En cuanto a los diagramas de interacción, significa que uno de los ejemplos de la Figura 16.15 podría ser útil.

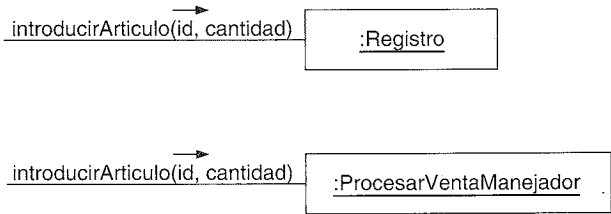


Figura 16.15. Opciones de Controlador.

En la elección de cuál de las clases es el controlador más apropiado influyen otros factores, que se estudiarán en la siguiente sección.

Durante el diseño, se asignan a una o más clases controlador las operaciones del sistema que se identificaron durante el análisis del comportamiento del sistema, como *Registro*, tal como se muestra en la Figura 16.16.

Los sistemas reciben eventos de entrada externos, normalmente a través de una GUI manejada por una persona. Otros medios de entrada pueden ser mensajes externos, como en un conmutador de telecomunicaciones para el procesamiento de llamadas, o señales desde sensores, como en sistemas de control de procesos.

Discusión

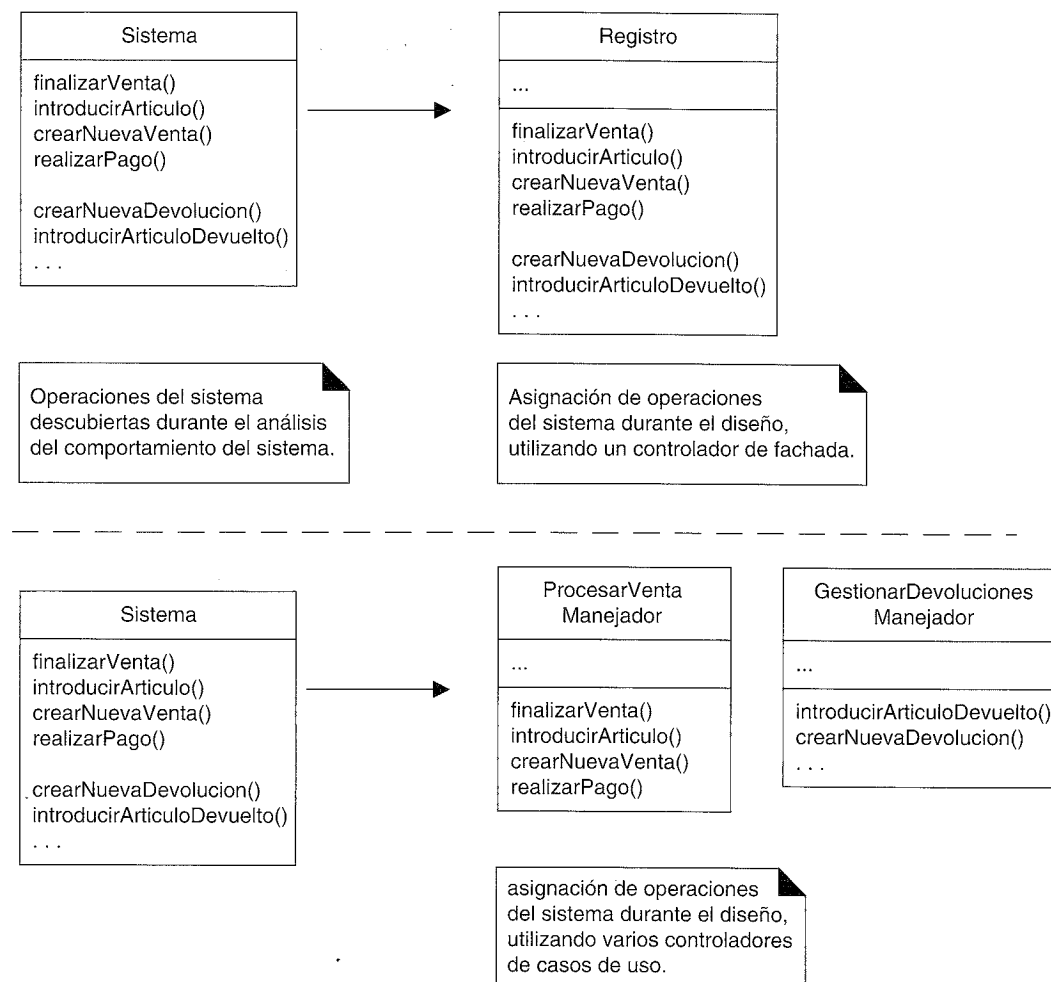


Figura 16.16. Asignación de las operaciones del sistema.

En todos los casos, si se utiliza un diseño de objetos, se debe escoger algún manejador para estos eventos. El patrón Controlador proporciona guías acerca de las opciones generalmente aceptadas y adecuadas. Como se ilustró en la Figura 16.14, el controlador es una especie de fachada en la capa del dominio para la capa de la interfaz.

A menudo, es conveniente utilizar la misma clase controlador para todos los eventos del sistema de un caso de uso de manera que es posible mantener la información acerca del estado del caso de uso en el controlador. Tal información es útil, por ejemplo, para identificar eventos del sistema que se apartan de la secuencia establecida (por ejemplo, una operación de *realizarPago* antes que la operación *finalizarVenta*). Podrían utilizarse diferentes controladores para casos de usos distintos.

Un error típico del diseño de los controladores es otorgarles demasiada responsabilidad.

Normalmente, un controlador debería *delegar* en otros objetos el trabajo que se necesita hacer; coordina o controla la actividad. No realiza mucho trabajo por sí mismo.

Por favor, diríjase a la sección “Cuestiones y Soluciones” que aparece más adelante para una discusión más elaborada.

La primera categoría de controlador es el controlador de fachada que representa al sistema global, dispositivo o subsistema. La idea es elegir algún nombre de clase que sugiera una cubierta, o fachada, sobre las otras capas de la aplicación, y que proporcione las llamadas a los servicios más importantes desde la capa de UI hacia las otras capas. Podría ser una abstracción de toda la unidad física, como un *Registro*⁶, *Conmutador-DeTelecomunicaciones*, *Telefono* o *Robot*; una clase que represente el sistema software completo, como un *SistemaPDV*, o cualquier otro concepto que el diseñador elija que represente al sistema o subsistema global, incluso, por ejemplo, *JuegoAjedrez* si fuera un software de juegos.

Los controladores de fachada son adecuados cuando no existen “demasiados” eventos del sistema, o no es posible que la interfaz de usuario (UI) redirija mensajes de los eventos del sistema a controladores alternativos, como un sistema de procesamiento de mensajes.

Si se elige un controlador de casos de uso, entonces hay un controlador diferente para cada caso de uso. Nótese que no es un objeto del dominio; es una construcción artificial para dar soporte al sistema (una *Fabricación Pura* en términos de los patrones GRASP). Por ejemplo, si la aplicación NuevaEra contiene casos de uso tales como *Procesar Venta* y *Gestionar Devoluciones*, entonces podría haber una clase *Procesar-VentaManejador* y así sucesivamente.

¿Cuándo se debería escoger un controlador de casos de uso? Es una alternativa a tener en cuenta cuando la asignación de las responsabilidades a un controlador de fachada nos conduce a diseños con baja cohesión o alto acoplamiento, generalmente cuando el controlador de fachada se está “inflando” con excesivas responsabilidades. Un controlador de casos de uso es una buena elección cuando hay muchos eventos del sistema repartidos en diferentes procesos; el controlador factoriza la gestión en clases separadas manejables, y también proporciona una base para conocer y razonar sobre el estado de los escenarios actuales en marcha.

En el UP y en el método más antiguo de Jacobson, Objectory [Jacobson92], existen los conceptos (opcionales) de clases frontera (*boundary*), control y entidad. Los **objetos frontera** son abstracciones de las interfaces, los **objetos entidad** son los objetos software del dominio independiente de la aplicación (y normalmente persistentes), y los **objetos control** son los manejadores de los casos de uso tal y como se describen en el patrón Controlador.

Un importante corolario del patrón Controlador es que los objetos interfaz (por ejemplo, los objetos ventana o elementos gráficos) y la capa de presentación no deberían ser responsables de llevar a cabo los eventos del sistema. En otras palabras, las operaciones del sistema se deberían manejar en la lógica de la aplicación o capas del dominio en lugar de en la capa de interfaz del sistema. Diríjase a la sección “Cuestiones y Soluciones” para ver un ejemplo.

⁶ Se utilizan varios términos para una unidad de PDV física, entre las que figuran: registro, terminal de punto de venta (TPDV), etcétera. A lo largo del tiempo, el “registro” ha llegado a encarnar la noción tanto de una unidad física, como la abstracción lógica de lo que registra ventas y pagos.

El objeto Controlador es normalmente un objeto del lado del cliente en el mismo proceso que la UI (por ejemplo, una aplicación con una GUI con las Swing de Java), entonces no es exactamente aplicable cuando el UI es un cliente Web en un navegador, y hay software del lado del servidor involucrado. En el último caso, existen varios patrones comunes para manejar los eventos del sistema que están fuertemente influenciados por el marco tecnológico escogido en el lado del servidor, como los servlets de Java. No obstante, es un estilo común crear controladores de casos de uso en el lado del servidor con o bien un servlet o un bean sesión EJB (*Enterprise JavaBeans*) para cada caso de uso. El objeto de sesión del lado del servidor representa una “sesión” de interacción con un actor externo.

Si la UI no es un cliente web (por ejemplo, es una GUI Swing o Windows), pero la aplicación invoca servicios remotos, es todavía común el uso del patrón Controlador. La UI reenvía la solicitud al Controlador local del lado del cliente, y el Controlador podría reenviar toda o parte de la gestión de la petición a los servicios remotos. Este diseño disminuye el acoplamiento entre la UI y los servicios remotos, y hace más fácil, por ejemplo, abastecer los servicios de manera local o remota, por medio de la indirección del Controlador del lado del cliente.

Resumiendo, el Controlador recibe la solicitud del servicio desde la capa de UI y coordina su realización, normalmente delegando a otros objetos.

Beneficios

- *Aumenta el potencial para reutilizar y las interfaces conectables (pluggable).* Asegura que la lógica de la aplicación *no* se maneja en la capa de interfaz. Técnicamente, las responsabilidades de un controlador podrían manejarse en un objeto interfaz, pero la implicación de tal diseño es que el código del programa y la lógica relacionada con la realización de la lógica de la aplicación estaría embebida en los objetos ventana o interfaz. Un diseño de una interfaz como controlador reduce la oportunidad de reutilizar la lógica en futuras aplicaciones, puesto que está ligada a una interfaz particular (por ejemplo, objetos ventana) que es raramente aplicable en otras aplicaciones. En cambio, delegando la responsabilidad de una operación del sistema a un controlador ayuda a la reutilización de la lógica en futuras aplicaciones. Y puesto que la lógica no está ligada a la capa de interfaz, puede sustituirse por una interfaz nueva.
- *Razonamiento sobre el estado de los casos de uso.* A veces es necesario asegurar que las operaciones del sistema tienen lugar en una secuencia válida, o ser capaces de razonar sobre el estado actual de la actividad y operaciones del caso de uso que está en marcha. Por ejemplo, podría ser necesario garantizar que la operación *realizarPago* no puede ocurrir hasta que haya tenido lugar la operación *finalizarVenta*. Si es así, es necesario capturar en algún sitio esta información de estado; el controlador es una opción razonable, especialmente si el mismo controlador se utiliza en todo el caso de uso (lo que se recomienda).

Cuestiones y Soluciones

Controladores saturados

Una clase controlador pobremente diseñada tendrá baja cohesión —no centrada en algo concreto y que gestiona demasiadas áreas de responsabilidad—; este controlador se denomina **controlador saturado**. Signos de que existe un controlador saturado son:

- Existe una *única* clase controlador que recibe *todos* los eventos del sistema en el sistema, y hay muchos. Esto ocurre a veces si se elige un controlador de fachada.
- El propio controlador realiza muchas de las tareas necesarias para llevar a cabo los eventos del sistema, sin delegar trabajo. Normalmente esto conlleva una violación de los patrones Experto en Información y Alta Cohesión.
- Un controlador tiene muchos atributos y mantiene información significativa sobre el sistema o el dominio, que debería haberse distribuido a otros objetos, o duplica información que se encuentra en otros sitios.

Hay varios remedios para un controlador saturado entre los que se encuentran:

1. Añadir más controladores: un sistema no tiene que tener sólo uno. En lugar de un controlador de fachada, utilice controladores de casos de uso. Por ejemplo, considere una aplicación con muchos eventos del sistema, como un sistema de reservas de vuelos.

Podría contener los siguientes controladores:

Controladores de casos de uso
RealizarReservaManejador
GestionarHorariosManejador
GestionarTarifasManejador

2. Diseñe el controlador de manera que, ante todo, delegue el cumplimiento de cada responsabilidad de una operación del sistema a otros objetos.

La capa de interfaz no maneja eventos del sistema

Reiterando: un corolario importante del patrón Controlador es que los objetos interfaz (por ejemplo, los objetos ventana) y la capa de interfaz, no deberían ser responsables de manejar los eventos del sistema. Como ejemplo, considere un diseño en Java que utiliza un *JFrame* para mostrar la información.

Asuma que la aplicación NuevaEra tiene una ventana que muestra la información de la venta y captura las operaciones del cajero. Utilizando el patrón Controlador, la Figura 16.17 ilustra una relación aceptable entre el *JFrame*, el Controlador y otros objetos en una parte del sistema de PDV (simplificado).

Nótese que la clase *JFrameVenta* —perteneciente a la capa de interfaz— pasa el mensaje *introducirArticulo* al objeto *Registro*. No se involucró en el procesamiento de la operación o en decidir cómo manejarla; la ventana sólo la delega a otra capa.

Asignando la responsabilidad de las operaciones del sistema a objetos en la capa de aplicación o del dominio —utilizando el patrón Controlador— en lugar de en la capa de interfaz incrementa el potencial para reutilizar. Si un objeto de la capa de interfaz (como el *JFrameVenta*) maneja una operación del sistema —que representa parte de un proceso de negocio— entonces la lógica del proceso del negocio estaría contenida en un

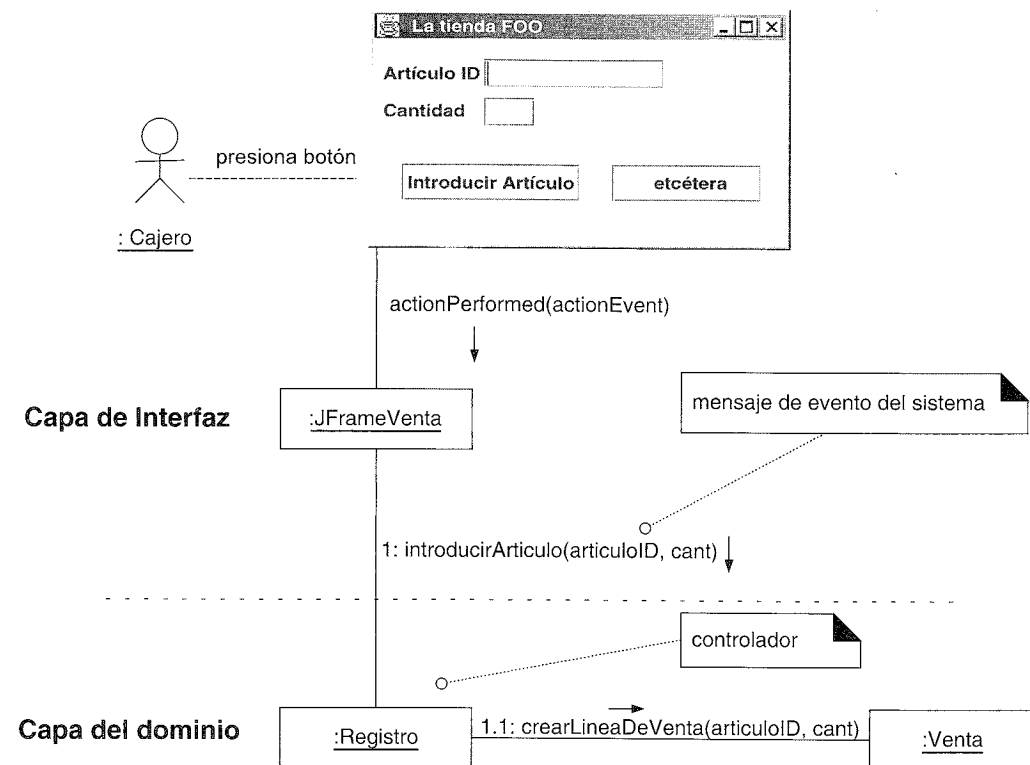


Figura 16.17. Acoplamiento deseable entre la capa de interfaz y la del dominio.

objeto interfaz (por ejemplo, del tipo ventana), por lo que la oportunidad para reutilizar es baja debido a su acoplamiento con una interfaz particular y con la aplicación.

En consecuencia, no es conveniente el diseño de la Figura 16.18.

La localización de la responsabilidad de las operaciones del sistema en un controlador que es un objeto del dominio facilita la reutilización de la lógica del programa lo que permite soportar el proceso de negocio asociado en futuras aplicaciones. También facilita la desconexión de la capa de interfaz y la utilización de un framework o tecnología de interfaz diferente o ejecutar el sistema en un modo “por lotes” sin conexión.

Sistemas de manejo de mensajes y el patrón *Command*

Algunas aplicaciones son sistemas de manejo de mensajes o servidores que reciben peticiones de otros procesos. Un conmutador de telecomunicaciones es un ejemplo típico. En tales sistemas, el diseño de la interfaz y el controlador es algo diferente. Los detalles se estudiarán en un capítulo posterior, pero en esencia, una solución típica es utilizar el patrón *Command* [GHJV95] y el patrón *Command Processor* [BMRSS96], que se presentarán en el Capítulo 34.

- **Command:** En un sistema de manejo de mensajes, cada mensaje podría representarse y manejarse mediante un objeto *Command* separado [GHJV95].
- **Fachada:** Un controlador de fachada es un tipo de Fachada (*Facade*) [GHJV95].

Patrones
Relacionados

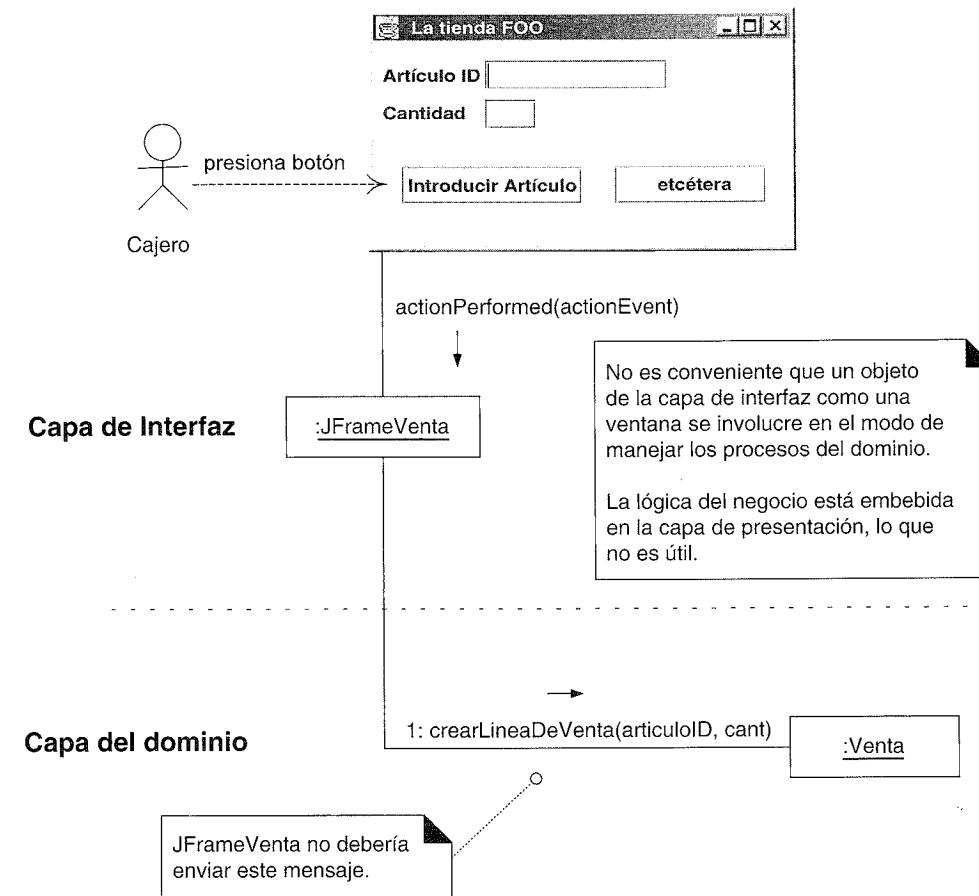


Figura 16.18. Acoplamiento menos conveniente entre la capa de interfaz y la del dominio.

- **Capas:** Es un patrón POSA [BMRSS96]. La ubicación de la lógica del dominio en la capa del dominio en lugar de en la capa de presentación forma parte del patrón de Capas (Layers).
- **Fabricación Pura:** Es otro patrón GRASP. Una Fabricación Pura es una creación arbitraria del diseñador, no una clase software cuyo nombre se inspira en el Modelo del Dominio. Un controlador de caso de uso es un tipo de Fabricación Pura.

16.11. Diseño de objetos y tarjetas CRC

Aunque formalmente no forma parte de UML, otro mecanismo que se utiliza algunas veces para ayudar a asignar responsabilidades e indicar las colaboraciones con otros objetos son las **tarjetas CRC** (tarjetas Clase-Responsabilidad-Colaborador) [BC89]. Kent Beck y Ward Cunningham fueron quienes promovieron el uso de estas tarjetas y son los principales responsables de estimular a los diseñadores de software a pensar de manera más abstracta en términos de asignación de responsabilidades y colaboraciones, y también del uso de los patrones.

Las tarjetas CRC son fichas, una por cada clase, en las que se escriben brevemente las responsabilidades de la clase, y una lista de los objetos con los que colabora para llevar a cabo esas responsabilidades. Se desarrollan normalmente en una sesión de trabajo en grupo pequeño. Los patrones GRASP se podrían aplicar cuando se tiene en cuenta el diseño mientras se utilizan las tarjetas CRC.

Las tarjetas CRC son una técnica para registrar los resultados de la asignación de responsabilidades y asignaciones. La información recopilada se puede enriquecer utilizando diagramas de clases y de interacción. Lo importante no son las tarjetas o los diagramas, sino tener presente la asignación de responsabilidades.

16.12. Lecturas adicionales

La metáfora de los objetos que colaboran con responsabilidades, o el **Diseño Dirigido por Responsabilidades**, surgió especialmente a partir del trabajo sobre objetos en Small-talk en Tektronix, Portland, de Kent Beck, Ward Cunningham, Rebecca Wirfs-Brock, y otros, que ha tenido gran influencia. El libro de texto que sirve de referencia es *Designing Object-Oriented Software* [WWW90], tan relevante hoy como cuando fue escrito.

Otros dos textos recomendados que destacan los principios fundamentales del diseño de objetos son *Object-Oriented Design Heuristics* de Riel y *Object Models* de Coad.

MODELO DE DISEÑO:
REALIZACIÓN DE CASOS DE USO
CON LOS PATRONES GRASP

Para inventar, necesitas una buena imaginación y un montón de trastos viejos.

Thomas Edison

Objetivos

- Diseñar las realizaciones de los casos de uso.
- Aplicar los patrones GRASP para asignar responsabilidades a las clases.
- Utilizar la notación de los diagramas de interacción UML para ilustrar el diseño de objetos.

Introducción

Este capítulo presenta cómo crear un diseño de objetos que colaboran con responsabilidades. Se presta especial atención a la aplicación de los patrones GRASP para desarrollar una solución bien diseñada. Por favor, obsérvese que los patrones GRASP como tales o por el nombre no son lo importante; son simplemente un apoyo al aprendizaje para ayudar a discutir y realizar de manera metódica el diseño de objetos fundamental.

Este capítulo expone los principios, utilizando el ejemplo del PDV NuevaEra, según los cuales un diseñador orientado a objetos asigna las responsabilidades y establece las interacciones entre los objetos —una técnica fundamental en el desarrollo orientado a objetos—.

Nota:

La asignación de responsabilidades y el diseño de colaboraciones son etapas muy importantes y creativas durante el diseño, mientras se elaboran los diagramas o mientras se programa.

El material es intencionalmente detallado; pretende ilustrar de manera exhaustiva que no existen decisiones “mágicas” o injustificadas en el diseño de objetos —la asignación de responsabilidades y la elección de las interacciones de objetos pueden explicarse y aprenderse de una forma racional—.

17.1. Realizaciones de casos de uso

Citando textualmente, “Una realización de caso de uso describe cómo se realiza un caso de uso particular en el modelo de diseño, en función de los objetos que colaboran” [RUP]. De manera más precisa, un diseñador puede describir el diseño de uno o más *escenarios* de un caso de uso; cada uno de estos se denomina una realización del caso de uso. La realización del caso de uso es un término o concepto del UP que se utiliza para recordarnos la conexión entre los requisitos expresados como casos de uso, y el diseño de objetos que satisface los requisitos.

Los diagramas de interacción UML son un lenguaje común para ilustrar las realizaciones de los casos de uso. Y como se estudió en el capítulo anterior, existen principios o patrones del diseño de objetos, como el Experto en Información y Bajo Acoplamiento, que se pueden aplicar durante este trabajo de diseño.

Como repaso, la Figura 17.20 (casi al final de este capítulo) expone las relaciones entre algunos artefactos del UP:

- El caso de uso sugiere los eventos del sistema que se muestran explícitamente en los diagramas de secuencia del sistema.
- Opcionalmente, podrían describirse los detalles de los efectos de los eventos del sistema en términos de los cambios de los objetos del dominio en los contratos de las operaciones del sistema.
- Los eventos del sistema representan los mensajes que inician los diagramas de interacción, los cuales representan el modo en el que los objetos interactúan para llevar a cabo las tareas requeridas —la realización del caso de uso—.
- Los diagramas de interacción comprenden la interacción de mensajes entre objetos software cuyos nombres se inspiran algunas veces en los nombres de las clases conceptuales del Modelo del Dominio, además de otras clases de objetos.

17.2. Comentarios sobre los artefactos

Diagramas de interacción y realizaciones de casos de uso

En la iteración actual estamos teniendo en cuenta varios escenarios y eventos del sistema tales como:

- *Procesar Venta*: *crearNuevaVenta*, *introducirArticulo*, *finalizarVenta*, *realizarPago*.

Si se utilizan los diagramas de interacción para representar las realizaciones de los casos de uso, se necesitará un diagrama de colaboración diferente para mostrar el manejo de cada mensaje de evento del sistema. Por ejemplo (Figura 17.1):

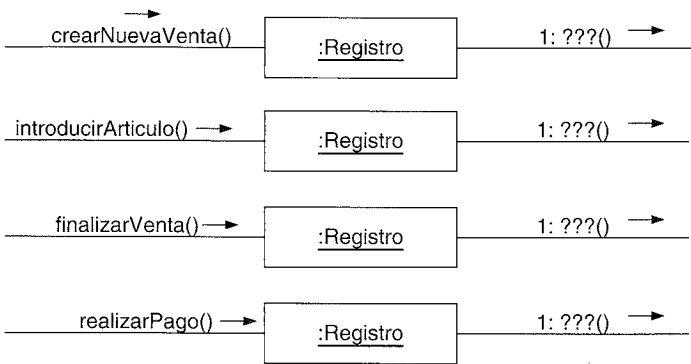


Figura 17.1. Diagramas de colaboración y manejo de los mensajes de eventos del sistema.

Por otro lado, si se utilizan los diagramas de secuencia, *podría* ser posible encajar todos los mensajes de eventos del sistema en el mismo diagrama, como en la Figura 17.2.

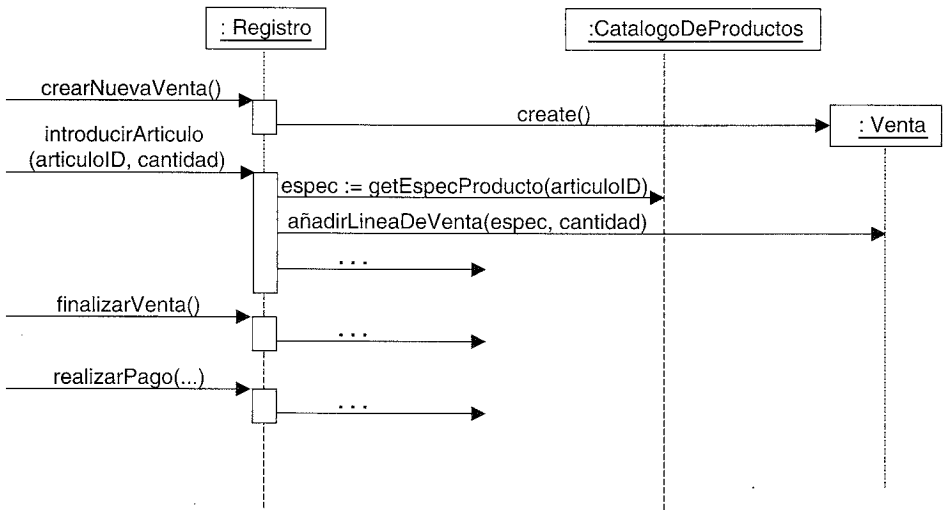


Figura 17.2. Un diagrama de secuencia y el manejo de los mensajes de eventos del sistema.

Sin embargo, ocurre a menudo que el diagrama de secuencia es entonces demasiado complejo o largo. Es legal, como con los diagramas de interacción, utilizar un diagrama de secuencia para cada mensaje de evento del sistema, como en la Figura 17.3.

Contratos y las realizaciones de casos de uso

Reiterando, podría ser posible diseñar las realizaciones de los casos de uso directamente a partir del texto de los casos de uso. Además, para algunas operaciones del sistema, se podrían haber escrito los contratos que añaden más detalles o son más específicos. Por ejemplo:

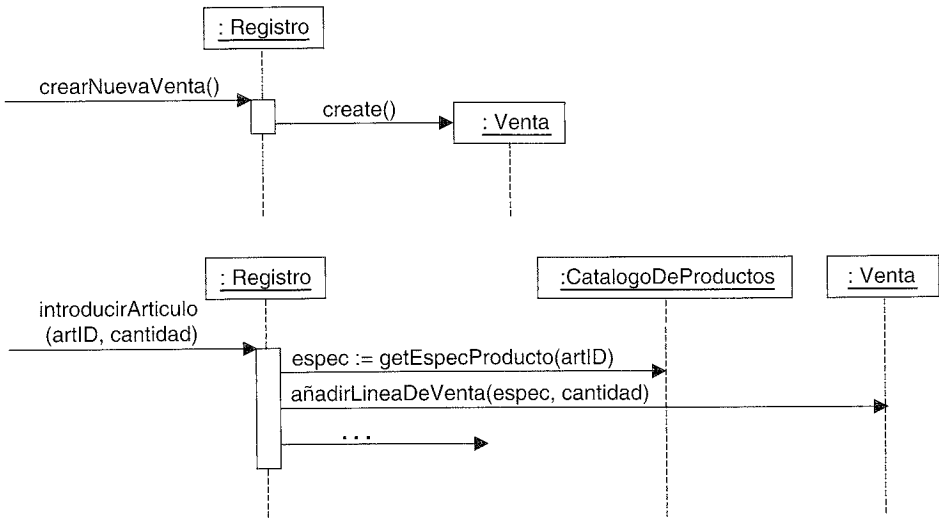


Figura 17.3. Múltiples diagramas de secuencia y manejo de los mensajes de eventos del sistema.

Contrato CO2: introducirArticulo

Operación:	introducirArticulo(articuloID:ArticuloID, cantidad:integer)
Referencias cruzadas:	Caso de Uso: Procesar Venta
Precondiciones:	Hay una venta en curso
Postcondiciones:	- Se creó una instancia de LineaDeVenta ldv (creación de instancias). - ...

Al mismo tiempo que tenemos en cuenta el texto de los casos de uso, para cada contrato, trabajamos cuidadosamente en expresar en el cambio de estado en las postcondiciones y diseñamos las interacciones para satisfacer los requisitos. Por ejemplo, dada esta operación parcial del sistema *introducirArticulo*, la Figura 17.4 muestra un diagrama de interacción parcial que satisface el cambio de estado de la creación de la instancia de *LineaDeVenta*.

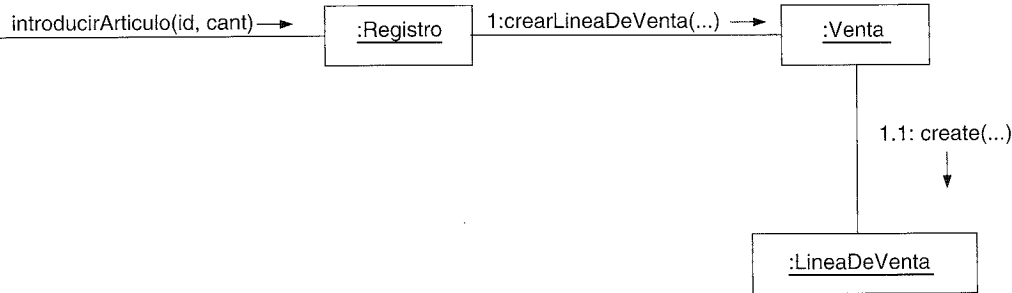


Figura 17.4. Diagrama de interacción parcial.

Advertencia: Los requisitos no son perfectos

Es útil tener presente que los casos de uso que se han escrito previamente y los contratos dan sólo una idea aproximada de lo que se tiene que conseguir. La historia del desarrollo del software nos va descubriendo invariablemente que los requisitos no son perfectos, o han cambiado. Esto no es una excusa para no tratar de hacer un buen trabajo de requisitos, sino el reconocimiento de que se necesita involucrar continuamente a los clientes y a los expertos en la materia en estudio para revisar y proporcionar retroalimentación sobre el comportamiento del sistema que se está desarrollando.

Una ventaja del desarrollo iterativo es que favorece de manera natural el descubrimiento de nuevos resultados del análisis y diseño durante el trabajo de diseño e implementación. El espíritu del desarrollo iterativo es capturar un grado “razonable” de información durante el análisis de requisitos, completando los detalles durante el diseño y la implementación.

El Modelo del Dominio y las realizaciones de los casos de uso

Algunos de los objetos software que interactúan mediante el paso de mensajes en los diagramas de interacción se inspiran en el Modelo del Dominio, como la clase conceptual *Venta* y la clase del diseño *Venta*. La elección de la asignación adecuada de la responsabilidad utilizando los patrones GRASP depende, en parte, de la información del Modelo del Dominio. Como ya se ha dicho, el Modelo del Dominio existente es probable que no sea perfecto; se espera que haya errores y omisiones. Se descubrirán nuevos conceptos que se olvidaron previamente, se ignorarán conceptos que se identificaron anteriormente, y lo mismo ocurrirá con las asociaciones y los atributos.

Clases conceptuales vs. clases del diseño

Recordemos que el Modelo del Dominio del UP no representa clases software, pero podría utilizarse para inspirar la presencia y los nombres de algunas clases software en el Modelo de Diseño. Durante la elaboración de los diagramas de interacción o la programación, los desarrolladores podrían mirar en el Modelo del Dominio para asignar los nombres a algunas clases del diseño; por tanto, se crea un diseño con un salto en la representación más bajo entre el diseño del software y nuestra percepción del dominio del mundo real con el que el software está relacionado (ver Figura 17.5).

¿Se deben limitar las clases del Modelo de Diseño a clases con nombres inspirados a partir del Modelo del Dominio? En absoluto, durante este trabajo de diseño es conveniente descubrir nuevas clases conceptuales que se obviaron durante el análisis de dominio inicial, y también crear clases software cuyos nombres y objetivos no estén relacionados en absoluto con el Modelo del Dominio.

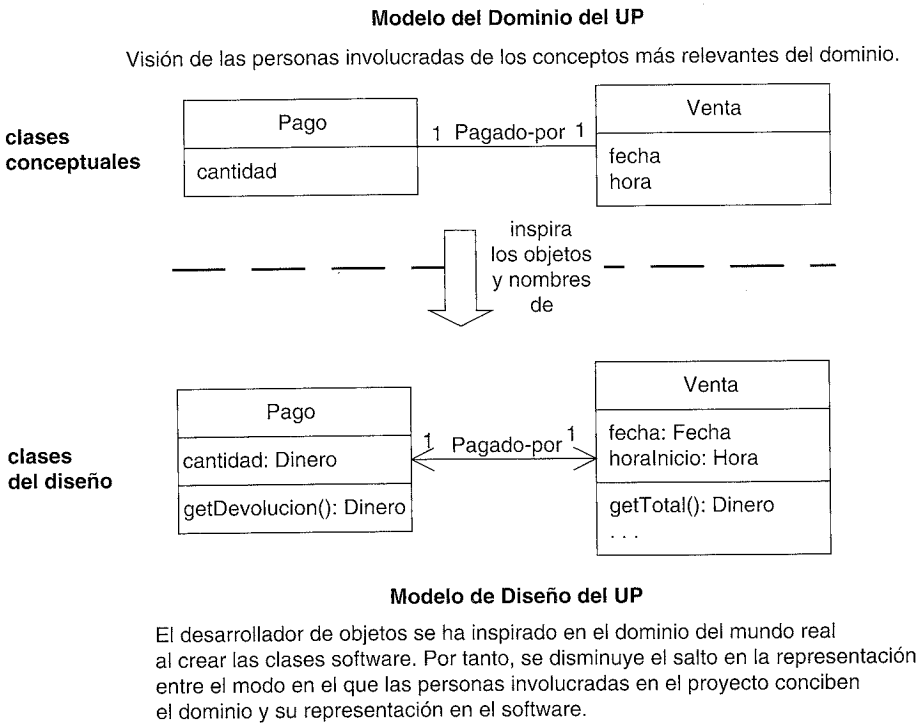


Figura 17.5. Disminución del salto en la representación nombrando las clases del diseño a partir de las clases conceptuales.

17.3. Realizaciones de casos de uso para la iteración de NuevaEra.

Las siguientes secciones exploran las elecciones y decisiones tomadas durante el diseño de una realización de caso de uso con objetos basado en los patrones GRASP. Las explicaciones son intencionalmente detalladas, en un intento de ilustrar que la creación de los diagramas de interacción bien diseñados no se trata de un “rollo” sin fundamento, sino que su construcción se basa en principios justificables.

En cuanto a la notación, el diseño de objetos para cada mensaje de eventos del sistema se representará en un diagrama independiente, para centrarse en las cuestiones de diseño de cada uno. Sin embargo, podrían haberse agrupado juntos en un único diagrama de secuencia.

17.4. Diseño de objetos: crearNuevaVenta

La operación del sistema *crearNuevaVenta* tiene lugar cuando un cajero solicita comenzar una nueva venta, después de que haya llegado un cliente con cosas que comprar. El caso de uso podría haber sido suficiente para decidir lo que era necesario, pero para este caso de estudio escribimos los contratos para todos los eventos del sistema, para que la explicación sea lo más completa posible.

Contrato CO1: crearNuevaVenta

Operación:	crearNuevaVenta()
Referencias cruzadas:	Caso de Uso: Procesar Venta
Precondiciones:	ninguna
Postcondiciones:	- Se creó una instancia de Venta v (creación de instancias). - v se asoció con el Registro (formación de asociaciones). - Se inicializaron los atributos de v.

Elección de la clase controlador

Nuestra primera elección de diseño comprende la elección del controlador para el mensaje de operación del sistema *crearNuevaVenta*. De acuerdo con el patrón Controlador, algunas de las opciones podrían ser:

representa el “sistema” global, dispositivo o subsistema	<i>Registro, SistemaPDV</i>
representa un receptor o manejador de todos los eventos del sistema de un escenario de caso de uso	<i>ProcesarVentaManejador, ProcesarVentaSesion</i>

Es aceptable elegir un controlador de fachada como el *Registro* si sólo hay unas pocas operaciones del sistema y el controlador no está asumiendo demasiadas responsabilidades (en otras palabras, si va a perder la cohesión). Es adecuado elegir un controlador de caso de uso cuando hay muchas operaciones del sistema y deseamos distribuir las responsabilidades con el fin de mantener cada clase controlador ligera y centrada (en otras palabras, cohesiva). En este caso, *Registro* será suficiente, puesto que sólo hay unas pocas operaciones del sistema.

Este *Registro* es un objeto software del Modelo de Diseño. No es un registro físico real sino una abstracción del software cuyo nombre se eligió para disminuir el salto en la representación entre nuestra concepción del dominio y el software.

Por tanto, el diagrama de interacción que se muestra en la Figura 17.6, comienza enviando el mensaje *crearNuevaVenta* al objeto software *Registro*.

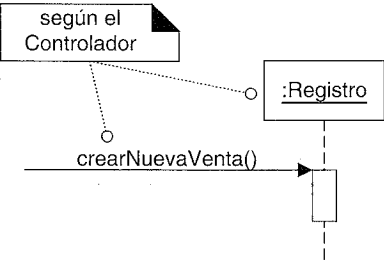


Figura 17.6. Aplicación del patrón GRASP Controlador.

Creación de una nueva Venta

Se debe crear un nuevo objeto software *Venta*, y el patrón GRASP Creador sugiere la asignación de la responsabilidad de creación a la clase que agrega, contiene o registra el objeto que se va a crear.

El análisis del Modelo del Dominio revela que se puede considerar que el *Registro* registra una *Venta*; de hecho, la palabra “registro” durante muchos años ha significado la cosa que graba (o registra) las transacciones contables, como las ventas.

Por tanto, el *Registro* es un candidato razonable para crear una *Venta*. Y consintiendo que el *Registro* cree la *Venta*, se puede asociar fácilmente a ella a lo largo del tiempo, de manera que durante futuras operaciones en la sesión, el *Registro* tendrá una referencia a la instancia de *Venta* actual.

Además de lo anterior, cuando se crea la *Venta*, se debe crear una colección vacía (contenedor, como una *List* de Java) para guardar todas las futuras instancias de *LineaDeVenta* que se añadirán. La instancia de *Venta* contendrá y mantendrá esta colección, lo que implica, según el Creador, que la *Venta* es una buena candidata para crearla.

Por tanto, el *Registro* crea la *Venta*, y la *Venta* crea una colección vacía, representada mediante un multiobjeto en el diagrama de interacción.

Por lo tanto, el diagrama de interacción de la Figura 17.7 ilustra el diseño.

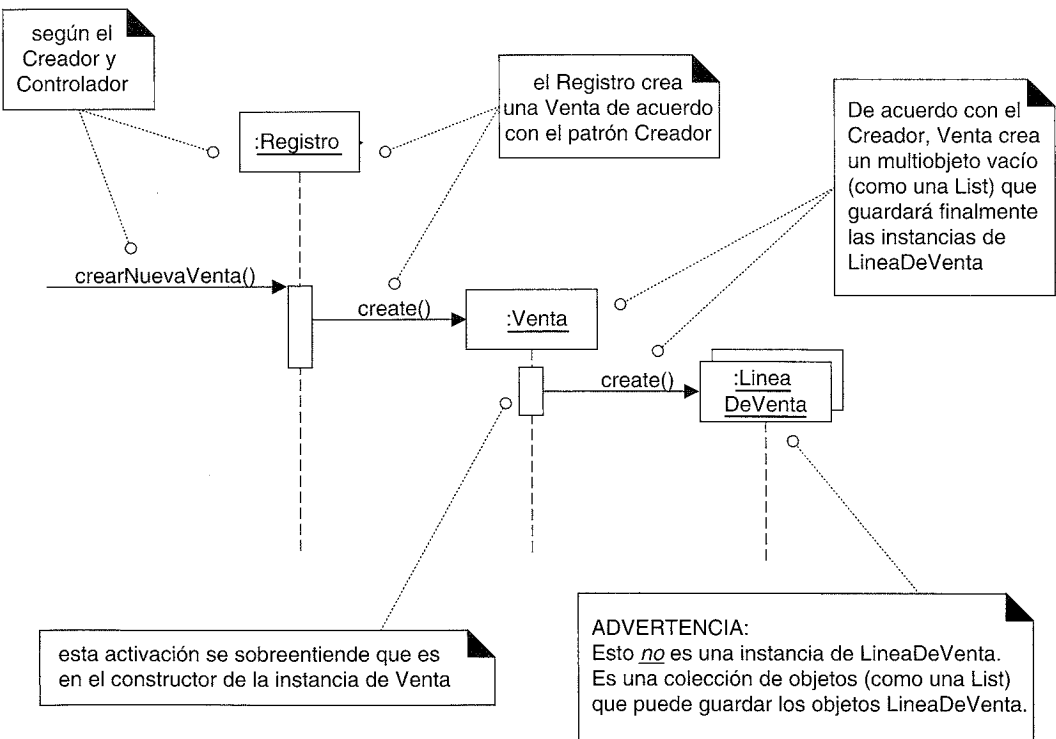


Figura 17.7. Creación de Venta y multiobjeto.

Conclusión

El diseño no fue difícil, pero lo importante de haberlo explicado de manera cuidadosa en términos del Controlador y el Creador era ilustrar que se pueden decidir y explicar, de manera racional y sistemática, los detalles de un diseño en base a principios y patrones, como los GRASP.

17.5. Diseño de objetos: introducirArticulo

La operación del sistema *introducirArticulo* tiene lugar cuando un cajero introduce el *articuloID* y (opcionalmente) la cantidad de ese artículo que se va a comprar. A continuación presentamos el contrato completo:

Contrato CO2: introducirArticulo

Operación:	introducirArticulo(articuloID:ArticuloID, cantidad:integer)
Referencias cruzadas:	Caso de Uso: Procesar Venta
Precondiciones:	Hay una venta en curso
Postcondiciones:	- Se creó una instancia de LineaDeVenta ldv (creación de instancias). - ldv se asoció con la Venta actual (formación de asociaciones). - ldv.cantidad pasó a ser cantidad (modificación de atributos). - ldv se asoció con una EspecificacionDelProducto, en base a la coincidencia del articuloID (formación de asociaciones).

Se construirá un diagrama de interacción que satisfaga la postcondición de *introducirArticulo*, utilizando los patrones GRASP para ayudar a tomar las decisiones de diseño.

Elección de la clase controlador

Nuestra primera elección tiene que ver con el manejo de la responsabilidad para el mensaje de operación del sistema *introducirArticulo*. Basado en el patrón Controlador, como para *crearNuevaVenta*, continuaremos utilizando el *Registro* como controlador.

¿Mostrar por pantalla la descripción y precio del artículo?

Debido a un principio de diseño denominado **Separación Modelo-Vista**, los objetos que no pertenecen a la GUI (como un *Registro* o *Venta*) no son responsables de involucrarse en las tareas de salida. Por tanto, aunque el caso de uso establezca que se muestran por pantalla la descripción y el precio después de la operación, el diseño lo ignorará en este momento.

Todo lo que se requiere con respecto a las responsabilidades de mostrar la información es que la información se conozca, lo que sucede en este caso.

Creación de una nueva LineaDeVenta

La postcondición del contrato *introducirArticulo* indica la creación, inicialización y asociación de una *LineaDeVenta*. El análisis del Modelo del Dominio revela que una *Venta* contiene objetos *LineaDeVenta*. Inspirándonos en el dominio, una *Venta* software podría contener igualmente objetos software *LineaDeVenta*. De aquí que, según el Creador, una *Venta* software sea una candidata adecuada para crear una *LineaDeVenta*.

La *Venta* puede asociarse con la *LineaDeVenta* recién creada almacenando la nueva instancia en su colección de líneas de venta. La postcondición indica que la nueva *LineaDeVenta*, cuando se crea, necesita una cantidad; por tanto, el *Registro* debe pasar esta cantidad a la *Venta*, quien debe pasarla como parámetro en el mensaje *create* (en Java, eso se implementaría como una llamada al constructor con un parámetro).

Por tanto, de acuerdo con el Creador, se envía a la *Venta* el mensaje *crearLineaDeVenta* para que cree una *LineaDeVenta*. La *Venta* crea una *LineaDeVenta*, y después almacena la nueva instancia en su colección permanente.

Los parámetros del mensaje *crearLineaDeVenta* incluyen la *cantidad*, de manera que la *LineaDeVenta* pueda registrarla, y del mismo modo la *EspecificacionDelProducto* que se corresponde con el *articuloID*.

Localización de una EspecificacionDelProducto

La *LineaDeVenta* necesita asociarse con la *EspecificacionDelProducto* que se corresponde con el *articuloID* de entrada. Esto implica que es necesario recuperar una *EspecificacionDelProducto*, en base a la coincidencia de un *articuloID*.

Antes de considerar *cómo* realizar la búsqueda, es conveniente considerar *quién* debe ser el responsable de ella. Por tanto, el primer paso es:

Comience asignando responsabilidades estableciendo claramente la responsabilidad.

Volviendo a plantear el problema:

¿Quién debe ser el responsable de conocer una *EspecificacionDelProducto*, basada en la coincidencia de un *articuloID*?

Esto no es ni un problema de creación ni uno sobre la elección de un controlador para un evento del sistema. Vamos a ver la primera aplicación del Experto en Información en el diseño.

En muchos casos, el patrón Experto es el principal patrón que se aplica. El Experto en Información sugiere que el objeto que tiene la información que se requiere para abordar la responsabilidad debería llevarla a cabo. ¿Quién conoce todos los objetos *EspecificacionDelProducto*?

El análisis del Modelo del Dominio revela que el *CatalogoDeProductos* lógicamente contiene todas las instancias *EspecificacionDelProducto*. Una vez más, inspirándonos en el dominio, diseñamos las clases software con una organización parecida: un *CatalogoDeProductos* software contendrá objetos software *EspecificacionDelProducto*.

Con eso decidido, entonces según el Experto en Información el *CatalogoDeProductos* es un buen candidato para esta responsabilidad de búsqueda puesto que conoce todos los objetos *EspecificacionDelProducto*.

Esto podría implementarse, por ejemplo, con un método denominado *getEspecificacion*¹.

Visibilidad del CatalogoDeProductos

¿Quién debería enviar el mensaje *getEspecificacion* al *CatalogoDeProductos* para solicitar una *EspecificacionDelProducto*?

Es razonable asumir que se crearon un *Registro* y un *CatalogoDeProductos* durante el caso de uso inicial *Poner en Marcha*, y que hay una conexión permanente desde el objeto *Registro* al objeto *CatalogoDeProductos*. Con este supuesto (que podríamos anotar en una lista de tareas de cosas que debemos asegurar en el diseño cuando vayamos a diseñar la inicialización), entonces es posible que el *Registro* envíe el mensaje *getEspecificacion* al *CatalogoDeProductos*.

Esto implica otro concepto en el diseño de objetos: la visibilidad. La **visibilidad** es la capacidad de un objeto de “ver” o tener una referencia a otro objeto.

Para que un objeto envíe un mensaje a otro objeto éste tiene que ser visible a aquél.

Puesto que asumiremos que el *Registro* tiene una conexión permanente —o referencia— al *CatalogoDeProductos*, éste es visible al *Registro* y, por tanto, puede enviarle un mensaje como el de *getEspecificacion*.

El siguiente capítulo abordará el tema de la visibilidad con más detenimiento.

Recuperación de objetos EspecificacionDelProducto de una base de datos

En la versión final de la aplicación del PDV NuevaEra, es improbable que todas las instancias de *EspecificacionDelProducto* se encuentren realmente en memoria. Será más probable que se almacenen en una base de datos relacional u objetual y se recuperarán bajo demanda; algunas podrían almacenarse en el proceso del cliente por motivos de rendimiento y tolerancia a fallos. Sin embargo, las cuestiones relacionadas con la recuperación de una base de datos se pospondrán por ahora en aras de la simplicidad. Se asumirá que todos los objetos *EspecificacionDelProducto* se encuentran en memoria.

El Capítulo 34 presentará el tema del acceso a base de datos de los objetos persistentes, que es un tema amplio en el que influye normalmente la elección de la tecnología, como J2EE, .NET, etcétera.

¹ La asignación de los nombres a los métodos de acceso es, por supuesto, una cuestión de estilo de cada lenguaje. Java siempre utiliza la forma *objeto.getFoo()*, C++ tiende a utilizar *objeto.foo()*, y C# utiliza *objeto.Foo*, lo que oculta (como Eiffel y Ada) si es una llamada a un método o un acceso directo a un atributo público. En los ejemplos se utiliza el estilo de Java.

El diseño de objetos de introducirArticulo

Dada la discusión anterior, el diagrama de interacción de la Figura 17.8 refleja las decisiones en cuanto a la asignación de responsabilidades y el modo en el que deberían interactuar los objetos. Obsérvese la considerable reflexión que se hizo para llegar a este diseño, basada en los patrones GRASP; el diseño de las interacciones entre objetos y la asignación de responsabilidades requiere alguna deliberación.

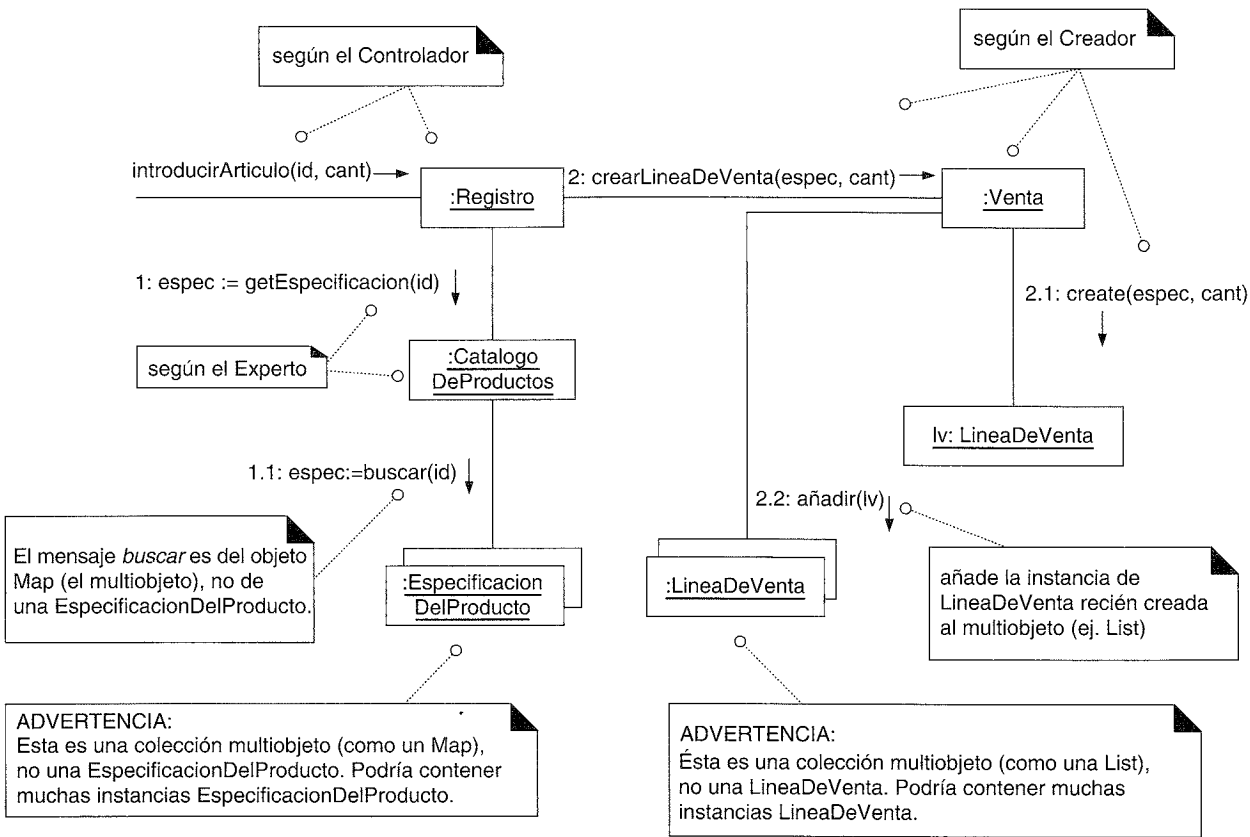


Figura 17.8. El diagrama de interacción para introducirArticulo.

Mensajes a los multiobjetos

Nótese que la interpretación en UML del envío de un mensaje a un multiobjeto es que se trata de un mensaje a la propia colección de objetos, en vez de una transmisión implícita a los miembros de la misma. Esto es especialmente obvio para las operaciones de colección genéricas como buscar y añadir.

Por ejemplo, en el diagrama de interacción de introducirArticulo:

- El mensaje buscar enviado al multiobjeto EspecificacionDelProducto es un mensaje que se envía una vez a la estructura de datos de la colección representada por el multiobjeto (como un Map de Java).
 - El mensaje genérico e independiente del lenguaje buscar se traducirá, durante la programación, para un lenguaje específico y librería. Quizás será finalmente

Map.get en Java. Podría haberse utilizado el mensaje get en los diagramas; se utilizó buscar para hacer entender que los diagramas de diseño podrían requerir alguna traducción a diferentes lenguajes y librerías.

- El mensaje añadir enviado al multiobjeto LineaDeVenta es para añadir un elemento a la estructura de datos de la colección representada por un multiobjeto (como una List de Java).

17.6. Diseño de objetos: finalizarVenta

La operación del sistema finalizarVenta tiene lugar cuando un cajero presiona un botón indicando el final de la venta. A continuación presentamos el contrato:

Contrato CO3: finalizarVenta

Operación:	finalizarVenta()
Referencias cruzadas:	Caso de Uso: Procesar Venta
Precondiciones:	Hay una venta en curso
Postcondiciones:	- Venta.esCompleta pasó a ser verdad (modificación de atributos).

Elección de la clase controlador

Nuestra primera elección tiene que ver con el manejo de la responsabilidad para el mensaje de operación del sistema finalizarVenta. Basado en el patrón GRASP Controlador, como para introducirArticulo, continuaremos utilizando el Registro como controlador.

Valor del atributo Venta.esCompleta

La postcondición del contrato establece que:

- Venta.esCompleta pasó a ser verdad (modificación de atributos).

Como siempre, el primer patrón a tener en cuenta debería ser el Experto, a menos que sea un problema de creación o del controlador (que no lo es).

¿Quién debería ser el responsable de poner el valor del atributo esCompleto de la Venta a verdad?

Según el Experto, debería ser la propia Venta, puesto que conoce y mantiene el atributo esCompleto. Por tanto, el Registro enviará un mensaje seHaCompletado a la Venta para asignarle el valor true.

Notación UML para mostrar las restricciones, notas y algoritmos

La Figura 17.9 muestra el mensaje seHaCompletado, pero no pone de manifiesto lo que ocurre en el método seHaCompletado (aunque en este caso se reconoce que es trivial). Algunas veces en UML deseamos utilizar texto para describir el algoritmo de un método, o especificar alguna restricción.

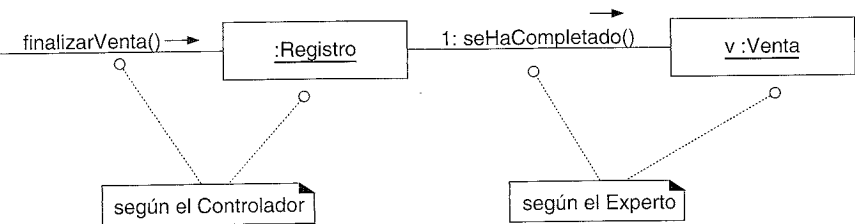


Figura 17.9. Finalización de la entrada de un artículo.

Para estas necesidades, UML proporciona tanto **restricciones** como **notas**. Una restricción UML es una información semánticamente significativa que se anexa a un elemento del modelo. Las restricciones en UML son texto encerrado entre llaves { }; por ejemplo, { x > 20 }. Se puede utilizar cualquier lenguaje formal o informal para las restricciones, y UML incluye especialmente **OCL** (lenguaje de restricciones de objetos) [WK99] si uno desea utilizarlo.

Una nota de UML es un comentario que no tiene impacto semántico, como la fecha de la creación o el autor.

Una nota siempre se muestra en un **cuadro de nota** (un cuadro de texto con la esquina doblada).

Una restricción podría mostrarse como un simple texto entre llaves, que es adecuado para declaraciones cortas. Sin embargo, las restricciones largas también podrían colocarse en un “cuadro de nota”, en cuyo caso el presunto cuadro de nota realmente contiene una restricción en lugar de una nota. El texto del cuadro va entre llaves para indicar que es una restricción.

En la Figura 17.10 se utilizan ambos estilos. Nótese que el estilo de restricción simple (entre llaves pero no en un cuadro) solamente muestra una sentencia que debe ser verdad (el significado clásico de una restricción en lógica). Por otro lado, la “restricción” en el cuadro de nota muestra la implementación del método Java de la restricción. Ambos estilos son legales para representar una restricción en UML.

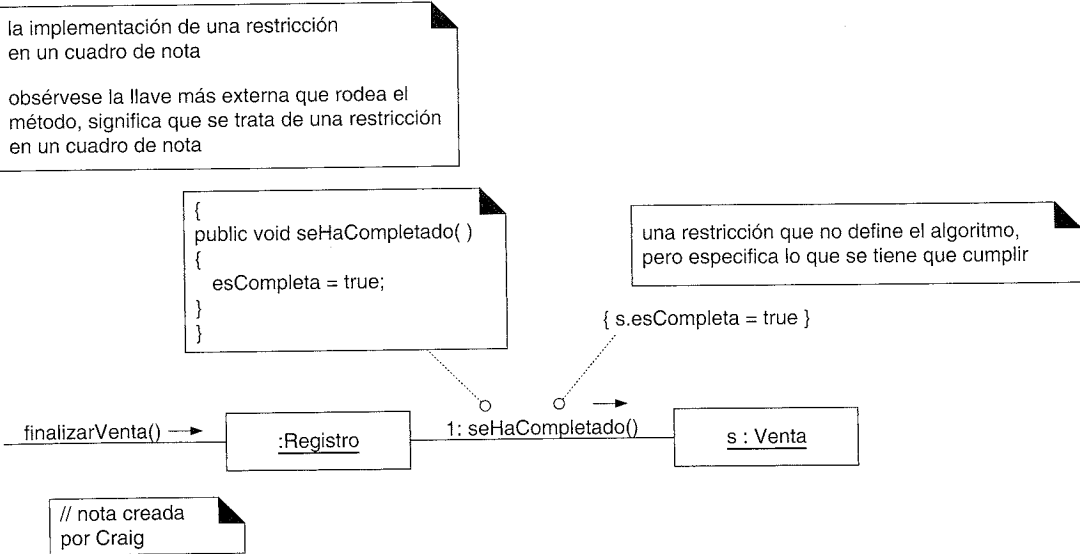


Figura 17.10. Restricciones y notas.

Cálculo del total de la Venta

Considere el siguiente fragmento del caso de uso *Procesar Venta*:

Escenario principal de éxito (o Flujo Básico):

- 1. El Cliente llega...
- 2. El Cajero le dice al Sistema que cree una nueva venta
- 3. El Cajero introduce el identificador del artículo
- 4. El Sistema registra la línea de la venta y ...
El Cajero repite los pasos 3-4 hasta que se indique
- 5. El Sistema presenta el total con los impuestos calculados.

En el paso 5, se presenta (o se muestra por pantalla) un total. Debido al principio de Separación Modelo-Vista, no deberíamos preocuparnos por el diseño de cómo se mostrará el total de la venta, pero es necesario asegurar que se conoce el total. Nótese que ninguna clase de diseño actualmente conoce el total de la venta, de manera que necesitamos crear un diseño de interacciones de objetos que satisfaga este requisito.

Como siempre, el Experto en Información debería ser un patrón a tener en cuenta a no ser que se trate de un problema de controlador o de creación (que no lo es).

Probablemente es obvio que la propia *Venta* deba ser la responsable de conocer su total, pero considere el siguiente análisis sólo con el fin de hacer el proceso de razonamiento para encontrar un Experto transparente como el cristal —con un ejemplo sencillo—.

- 1. Establezca la responsabilidad:
 - o ¿Quién debe ser el responsable de conocer el total de la venta?
- 2. Reúna la información necesaria:
 - o El total de la venta es la suma de los subtotales de todas las líneas de venta.
 - o El subtotal de la línea de venta := cantidad de la línea de venta * precio de la descripción del producto.
- 3. Liste la información requerida para abordar esta responsabilidad y las clases que conocen esta información.

Información requerida para el total de la Venta	Experto en Información
EspecificacionDelProducto.precio	EspecificacionDelProducto
LineaDeVenta.cantidad	LineaDeVenta
Todas las LineasDeVenta de la Venta actual	Venta

A continuación presentamos un análisis detallado:

- ¿Quién deber ser responsable del cálculo del total de la *Venta*? Según el Experto, debería ser la propia *Venta*, puesto que conoce todas las instancias de *LineaDeVenta* cuyos subtotales se deben sumar para calcular el total de la venta. Por tanto, la *Venta* tendrá la responsabilidad de conocer su total, implementada como un método *getTotal*.

- Para que una *Venta* calcule su total, necesita el subtotal de cada *LineaDeVenta*. ¿Quién debe ser responsable de crear el subtotal de la *LineaDeVenta*? Según el Experto, debería ser la propia *LineaDeVenta*, puesto que conoce la cantidad y la *EspecificacionDelProducto* con la que está asociada. Por tanto, la *LineaDeVenta* tendrá la responsabilidad de conocer su subtotal, implementada como un método *getSubtotal*.
- Para que una *LineaDeVenta* calcule su subtotal, necesita el precio de la *EspecificacionDelProducto*. ¿Quién debería ser responsable de proporcionar el precio de la *EspecificacionDelProducto*? Según el Experto, debería ser la propia *EspecificacionDelProducto*, puesto que encapsula el precio como un atributo. Por tanto, la *EspecificacionDelProducto* tendrá la responsabilidad de conocer su precio, implementada como una operación *getPrecio*.

Aunque el análisis anterior es trivial en este caso, y el atormentador grado de elaboración presentado está fuera de lugar para el diseño que nos ocupa, la misma estrategia de razonamiento para encontrar un Experto puede y debe aplicarse en situaciones más difíciles. Descubrirá que una vez que aprenda estos principios puede hacer rápidamente esta clase de razonamiento mentalmente.

El diseño de *Venta--getTotal*

Dada la anterior discusión, es conveniente ahora construir un diagrama de interacción que ilustre lo que ocurre cuando se envía el mensaje *getTotal* a la *Venta*. El primer mensaje de este diagrama es *getTotal*, pero observe que el mensaje *getTotal* no es un evento del sistema.

Eso nos lleva a la siguiente observación:

No todos los diagramas de interacción comienzan con un mensaje de evento del sistema; pueden comenzar con cualquier mensaje para el que el diseñador desee mostrar las interacciones.

El diagrama de interacción se muestra en la Figura 17.11. Primero se envía el mensaje *getTotal* a la instancia de *Venta*. La *Venta* entonces enviará un mensaje *getSubtotal* a cada instancia de *LineaDeVenta* relacionada. La *LineaDeVenta* enviará sucesivamente un mensaje *getPrecio* a las instancias *EspecificacionDelProducto* asociadas.

Puesto que la aritmética (usualmente) no se ilustra mediante mensajes, los detalles de los cálculos se pueden ilustrar adjuntando al diagrama algoritmos o restricciones que definen los cálculos.

¿Quién enviará el mensaje *getTotal* a la *Venta*? Lo más probable es que sea un objeto de la capa de UI, como un *JFrame* de Java.

Obsérvese en la Figura 17.12 el uso de las notas de algoritmos y restricciones, para exponer los detalles de *getTotal* y *getSubtotal*.

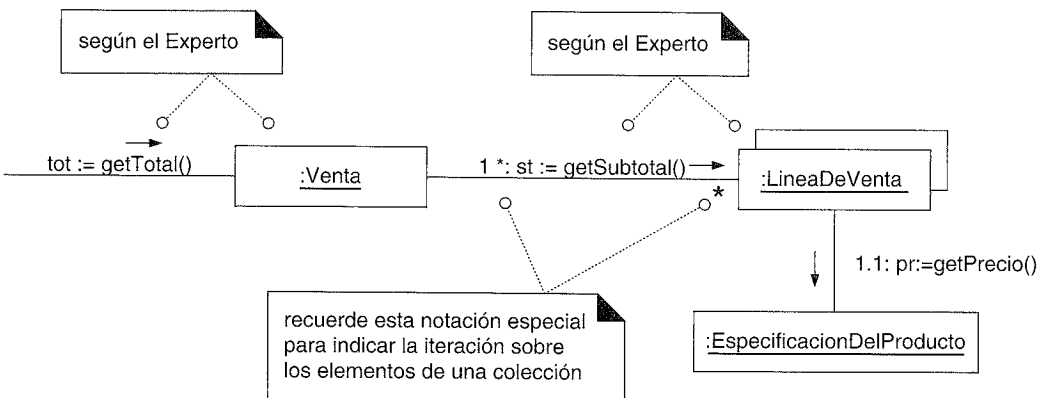


Figura 17.11. Diagrama de interacción *Venta--getTotal*.

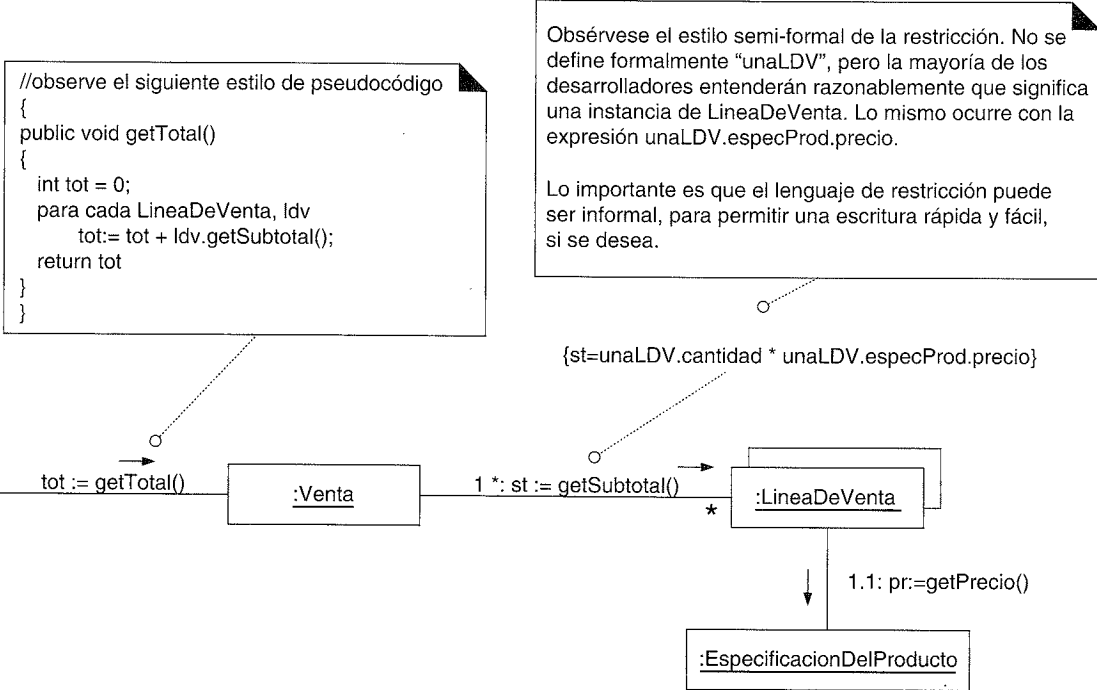


Figura 17.12. Notas de algoritmos y restricciones.

17.7. Diseño de objetos: realizarPago

La operación del sistema *realizarPago* tiene lugar cuando un cajero introduce la cantidad de dinero entregada para el pago. A continuación presentamos el contrato completo:

Contrato CO4: realizarPago

Operación: realizarPago(cantidad: Dinero)
Referencias cruzadas: Caso de Uso: Procesar Venta

Precondiciones:	Hay una venta en curso
Postcondiciones:	- Se creó una instancia de Pago p (creación de instancias). - p.cantidadEntregada pasó a ser cantidad (modificación de atributos). - p se asoció con la Venta actual (formación de asociaciones). - La Venta actual se asoció con la Tienda (formación de asociaciones); (para añadirlo al registro histórico de las ventas completadas).

Se construirá un diseño que satisfaga la postcondición de *realizarPago*.

Creación del Pago

Una de las postcondiciones del contrato establece:

- Se creó una instancia de *Pago p* (creación de instancias).

Ésta es una responsabilidad de creación, de manera que se debería aplicar el patrón GRASP Creador.

¿Quién registra, agrega, utiliza más estrechamente, o contiene un *Pago*? Existe un cierto atractivo en establecer que un *Registro* lógicamente registra un *Pago*, porque en el dominio real un “registro” recopila la información contable, luego es un candidato según el objetivo de reducir el salto en la representación en el diseño del software. Adicionalmente, es razonable esperar que una *Venta* software utilizará estrechamente un *Pago*; por tanto, podría ser una candidata.

Otra forma de encontrar un creador es utilizar el patrón Experto en función de quién es el Experto en Información con respecto a los datos de inicialización —la cantidad entregada en este caso—. El *Registro* es el controlador que recibe el mensaje de la operación del sistema *realizarPago*, luego, inicialmente tendrá la cantidad entregada. En consecuencia, el *Registro* es de nuevo un candidato.

Resumiendo, hay dos candidatos:

- *Registro*.
- *Venta*.

Ahora, esto nos lleva a una idea de diseño clave:

Cuando hay elecciones de diseño alternativas, mire detenidamente las implicaciones en cuanto a la cohesión y el acoplamiento de las alternativas, y posiblemente también considere las futuras presiones de evolución de las alternativas. Elija una alternativa con buena cohesión, acoplamiento y estabilidad ante posibles cambios futuros.

Considere algunas de las implicaciones de estas elecciones en función de los patrones GRASP Alta Cohesión y Bajo Acoplamiento. Si se elige la *Venta* para crear el *Pago*, es más ligero el trabajo (o responsabilidades) del *Registro* —dando lugar a una defini-

ción de *Registro* más simple—. También, el *Registro* no necesita conocer la existencia de una instancia de *Pago* porque se puede registrar indirectamente por medio de la *Venta* —lo que produce una disminución del acoplamiento en el *Registro*—. Esto nos lleva al diseño que se muestra en la Figura 17.13.

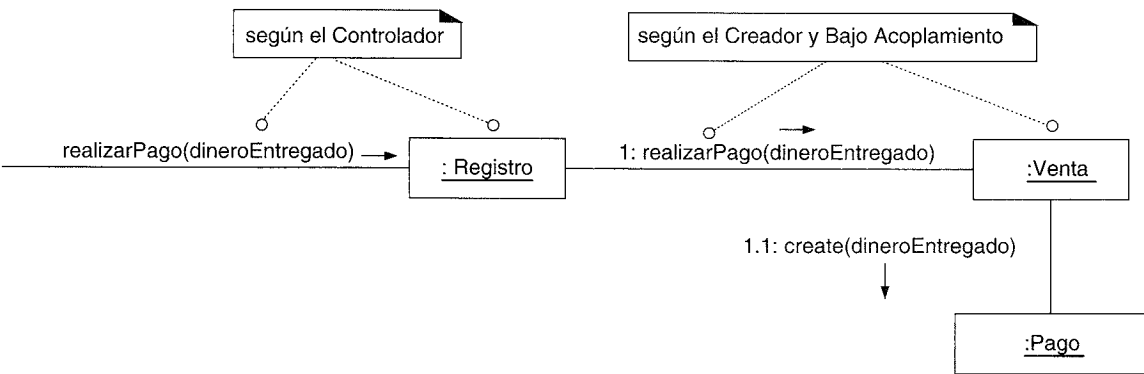


Figura 17.13. Diagrama de interacción Registro--realizarPago.

Este diagrama de interacción satisface la postcondición del contrato: se ha creado el *Pago*, asociado con la *Venta*, y se asigna el valor a su *cantidadEntregada*.

Registro de una Venta

Una vez completada, los requisitos establecen que la venta se debería colocar en un registro de históricos. Como siempre, el Experto en Información debería ser uno de los primeros patrones a tener en cuenta a menos que sea un problema de controlador o de creación (que no lo es), y se debería establecer la responsabilidad:

¿Quién es el responsable de conocer todas las ventas registradas y hacer el apunte en el registro?

Según el objetivo de salto en la representación bajo en el diseño del software (en relación con nuestros conceptos del dominio) es razonable que una *Tienda* conozca todas las ventas registradas, puesto que están fuertemente relacionadas con sus asuntos financieros. Otras alternativas comprenden conceptos clásicos de contabilidad, como *LibroMayorDeVentas*. Tiene sentido utilizar un objeto *LibroMayorDeVentas* cuando el diseño crece y la tienda pierde la cohesión (ver Figura 17.14).

Nótese también que la postcondición del contrato indica que se relacione la *Venta* con la *Tienda*. Éste es un ejemplo donde las postcondiciones podría no ser lo que realmente queremos conseguir en el diseño. Quizás no pensamos en el *LibroMayorDeVentas* antes, pero ahora que lo tenemos, elegimos usarlo en lugar de una *Tienda*. Si éste fuera el caso, idealmente se añadiría también el *LibroMayorDeVentas* al Modelo del Dominio, ya que es un nombre de un concepto en el dominio del mundo real. Son de esperar este tipo de descubrimientos y cambios durante el trabajo de diseño.

En este caso, nos mantendremos fieles al plan original de utilizar la *Tienda* (ver Figura 17.15).

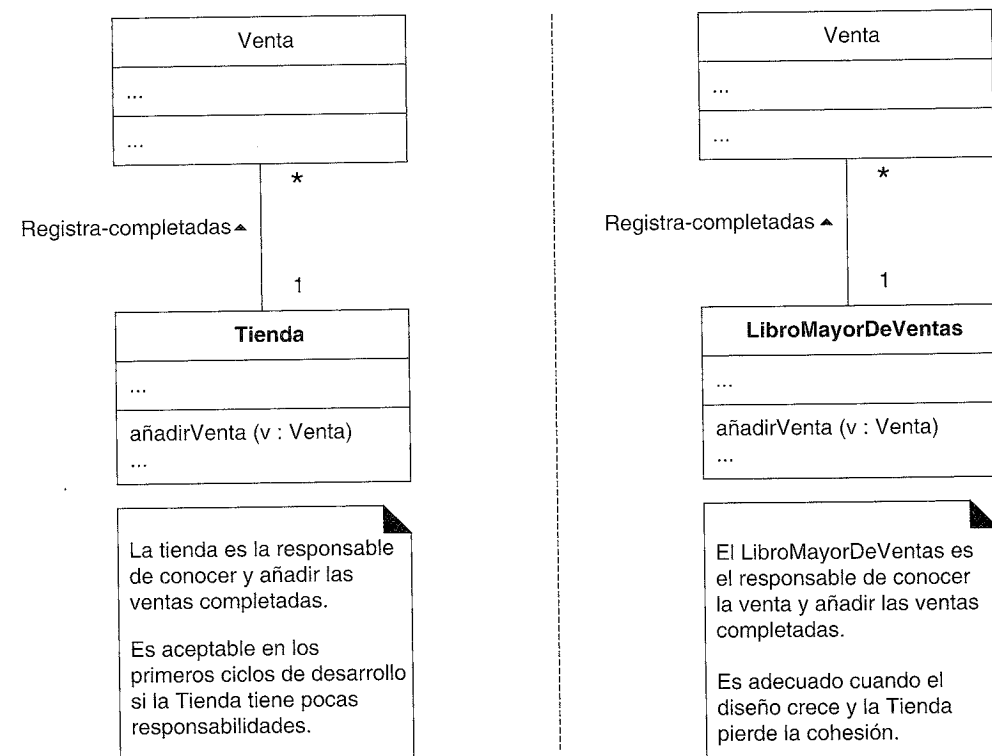


Figura 17.14. ¿Quién debería ser responsable de conocer la venta completada?

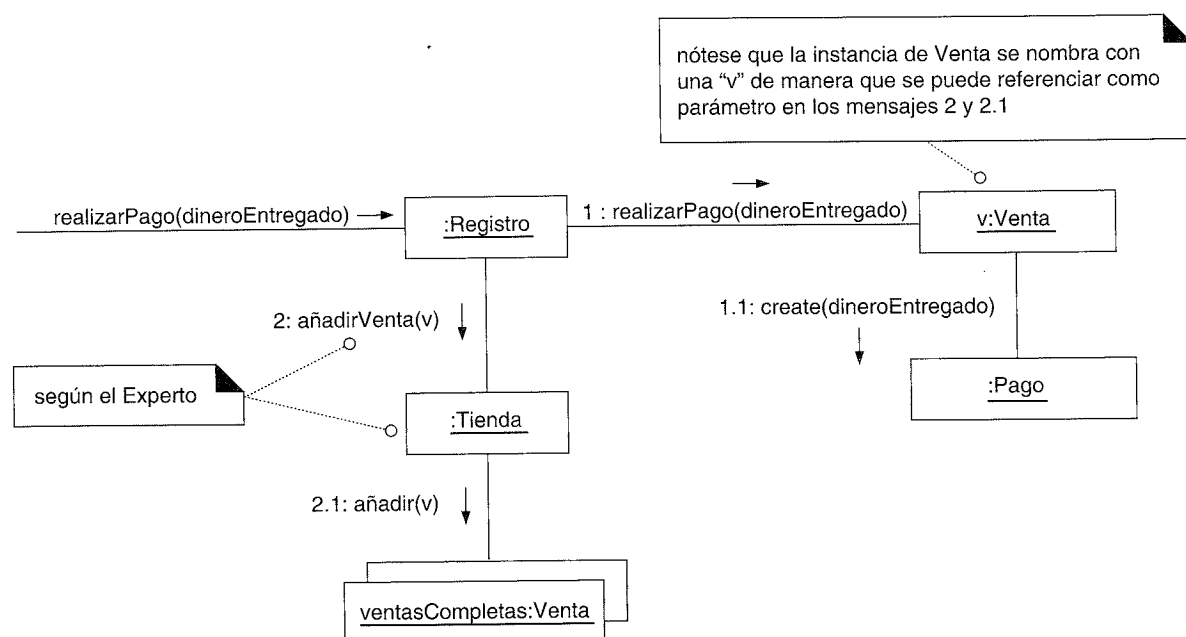


Figura 17.15. Registro de una venta completada.

Cálculo de la devolución

El caso de uso *Procesar Venta* implica que se imprima el saldo deudor a partir de un pago en un recibo y se muestre por pantalla de algún modo.

Debido al principio de Separación Modelo-Vista, no deberíamos preocuparnos por el modo en el que se visualizará o imprimirá el dinero que hay que devolver, pero es necesario asegurar que se conoce. Nótese que ninguna clase conoce actualmente la devolución, de manera que necesitamos crear un diseño de interacciones de objetos que satisfaga estos requisitos.

Como siempre, se debería tener en cuenta el Experto en Información a menos que sea un problema de controlador o creación (que no lo es), y se debería establecer la responsabilidad:

¿Quién es el responsable de conocer la devolución?

Para calcular la devolución, se requiere el total de la venta y el dinero en efectivo entregado. Por tanto, la *Venta* y el *Pago* son Expertos parciales en la solución del problema.

Si el *Pago* es ante todo responsable de conocer la devolución, necesitará tener visibilidad de la *Venta*, para pedirle su total. Puesto que actualmente no sabe nada de la *Venta*, este enfoque incrementa el acoplamiento global en el diseño —no soportaría el patrón de Bajo Acoplamiento—.

En cambio, si la *Venta* es principalmente responsable de conocer la devolución, necesita tener visibilidad del *Pago*, para solicitarle el dinero en efectivo entregado. Puesto que la *Venta* ya tiene visibilidad del *Pago* —como su creador— este enfoque no incrementa el acoplamiento global y, por tanto, es preferible este diseño.

En consecuencia, el diagrama de interacción de la Figura 17.16 proporciona una solución para conocer la cantidad a devolver.

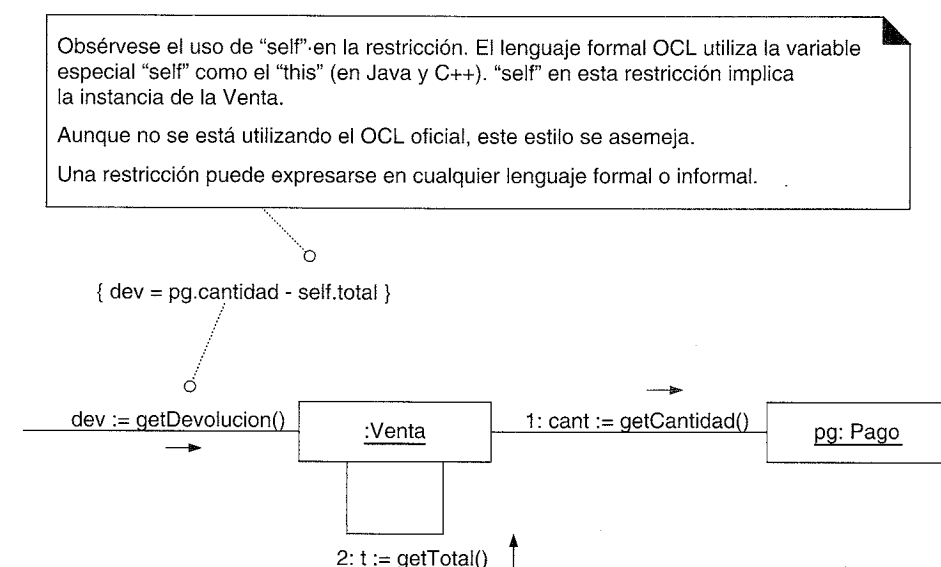


Figura 17.16. Diagrama de interacción Venta--getDevolucion.

17.8. Diseño de objetos: ponerEnMarcha

¿Cuándo crear el diseño de ponerEnMarcha?

La mayoría, si no todos, los sistemas tienen el caso de uso *ponerEnMarcha*, y alguna operación del sistema inicial relacionada con el comienzo de la aplicación. Aunque esta operación del sistema *ponerEnMarcha* es la primera que se va a ejecutar, posponga el desarrollo de un diagrama de interacción para ella hasta que se hayan tenido en cuenta todas las otras operaciones del sistema. Esto asegura que se haya descubierto la información relativa a las actividades de inicialización que se requieren para soportar los diagramas de interacción de operaciones del sistema posteriores.

Haga el diseño de la inicialización en último lugar.

Cómo comienzan las aplicaciones

La operación *ponerEnMarcha* representa de manera abstracta la fase de inicialización de la ejecución cuando se lanza una aplicación. Para entender cómo diseñar un diagrama de interacción para esta operación, es útil entender el contexto en el que puede ocurrir la inicialización. El modo en el que una aplicación comienza y se inicializa depende del lenguaje de programación y del sistema operativo.

En todos los casos, un estilo de diseño común es crear en último término un **objeto del dominio inicial**, que es el primer objeto software del “dominio” que se crea.

Una nota sobre la terminología: como se verá, las aplicaciones se organizan en capas lógicas que separan los aspectos más importantes de la aplicación. Esto comprende la capa de UI (para las cuestiones de UI) y una capa del “dominio” (para cuestiones de la lógica del dominio). La capa del dominio del Modelo de Diseño está formada por las clases software cuyos nombres están inspirados en el vocabulario del dominio, y que contienen la lógica de la aplicación. Prácticamente todos los objetos del diseño que hemos considerado, como *Venta* y *Registro*, son objetos del dominio en la capa del dominio del Modelo de Diseño.

El objeto de dominio inicial, una vez creado, es el responsable de la creación de los objetos del dominio que son sus “hijos”. Por ejemplo, si se elige una *Tienda* como el objeto del dominio inicial, podría ser el responsable de la creación de un objeto *Registro*.

El lugar donde se crea este objeto del dominio inicial depende de la tecnología de objetos escogida. Por ejemplo, en una aplicación Java, podría crearlo el método *main*, o delegar el trabajo al objeto *factoría* que lo crea.

```
public class Main
{
    public static void main(String [] args)
    {
        //La Tienda es el objeto del dominio inicial.
        //La Tienda crea algún otro objeto del dominio.
```

```
Tienda tienda = new Tienda();
Registro registro = tienda.getRegistro();
JFrameProcesarVenta frame = new JFrameProcesarVenta(registro);
...
}
```

Interpretación de la operación del sistema ponerEnMarcha

La discusión anterior ilustra que la operación del sistema *ponerEnMarcha* es una abstracción independiente del lenguaje. Durante el diseño, existen variaciones en cuanto al lugar de creación del objeto inicial, y si controla o no el proceso. El objeto del dominio inicial no suele tomar el control si se trata de una GUI; en otro caso, lo hace con frecuencia.

Los diagramas de operación para la operación *ponerEnMarcha* representan lo que ocurre cuando se crea el objeto inicial del dominio del problema, y opcionalmente lo que sucede si toma el control. No incluyen ninguna actividad anterior o siguiente de los objetos en la capa de GUI, si existe alguno.

Por tanto, la operación *ponerEnMarcha* puede reinterpretarse como:

1. En un diagrama de interacción, envíe un mensaje *create()* para crear el objeto de dominio inicial.
2. (opcional) Si el objeto inicial toma el control del proceso, en un segundo diagrama de interacción, envíe el mensaje *ejecutar* (o algo equivalente) al objeto inicial.

La operación PonerEnMarcha de la aplicación del PDV

La operación del sistema *ponerEnMarcha* tiene lugar cuando un responsable de la tienda enciende el sistema de PDV y se carga el software. Asuma que el objeto del dominio inicial *no* es responsable de controlar el proceso; el control permanecerá en la capa de UI (como un *JFrame* de Java) después de que se cree el objeto del dominio inicial. Por tanto, el diagrama de interacción para la operación *ponerEnMarcha* podría reinterpretarse únicamente como el envío del mensaje *create()* para crear el objeto inicial.

Elección del objeto del dominio inicial

¿Cuál debería ser la clase del objeto del dominio inicial?

Elija como objeto del dominio inicial la clase de la raíz de la jerarquía de agregación o contención, o cercana a ella. Esto podría ser un controlador de fachada, como un *Registro*, o algún otro objeto que se considera que contiene todos o la mayoría de los objetos, como una *Tienda*.

Las consideraciones de Alta Cohesión y Bajo Acoplamiento podrían influir en la elección entre las alternativas. En esta aplicación, se elige la *Tienda* como el objeto inicial.

Objetos persistentes: EspecificacionDelProducto

Las instancias de *EspecificacionDelProducto* residirán en un medio de almacenamiento persistente, como una base de datos relacional u objetual. Durante la operación *ponerEnMarcha*, si sólo hay unos pocos de estos objetos, se podrían cargar todos en la memoria principal del ordenador. Sin embargo, si hay muchos, cargarlos todos consumiría demasiada memoria o tiempo. Alternativamente —y más probable— se cargarán en memoria bajo demanda las instancias individuales cuando se requieran.

El diseño de la manera de cargar dinámicamente bajo demanda los objetos desde una base de datos a memoria es sencilla si se utiliza una base de datos objetual, pero difícil para una base de datos relacional. Este problema se pospone por ahora y se asume que todas las instancias de *EspecificacionDelProducto* pueden ser creadas en memoria “mágicamente” por el objeto *CatalogoDeProductos*.

El Capítulo 34 estudia el problema de los objetos persistentes y el modo de cargarlos en memoria.

Diseño de Tienda--create()

Las tareas de creación e inicialización se derivan a partir de las necesidades del trabajo de diseño anterior, como el diseño de la gestión de *introducirArticulo*, etcétera. Reflexionando sobre los diseños de interacciones previos, se puede identificar el siguiente trabajo de inicialización:

- Se necesita crear una *Tienda*, *Registro*, *CatalogoDeProductos* y objetos *EspecificacionDelProducto*.
- Se necesitan asociar los objetos *EspecificacionDelProducto* con el *CatalogoDeProductos*.
- Se necesita asociar la *Tienda* con el *CatalogoDeProductos*.
- Se necesita asociar la *Tienda* con el *Registro*.
- Se necesita asociar el *Registro* con el *CatalogoDeProductos*.

La Figura 17.17 muestra un diseño. Se escogió la *Tienda* para crear el *CatalogoDeProductos* y el *Registro* según el patrón Creador. Del mismo modo, se eligió el *CatalogoDeProductos* para crear los objetos *EspecificacionDelProducto*. Recuerde que este enfoque de creación de las especificaciones es temporal. En el diseño final, se materializará desde una base de datos, cuando sea necesario.

Notación UML: Obsérvese que la creación de todas las instancias de *EspecificacionDelProducto* y su inclusión en un contenedor tienen lugar en una sección de repetición, que se indica con el “*” a continuación de los números de secuencia.

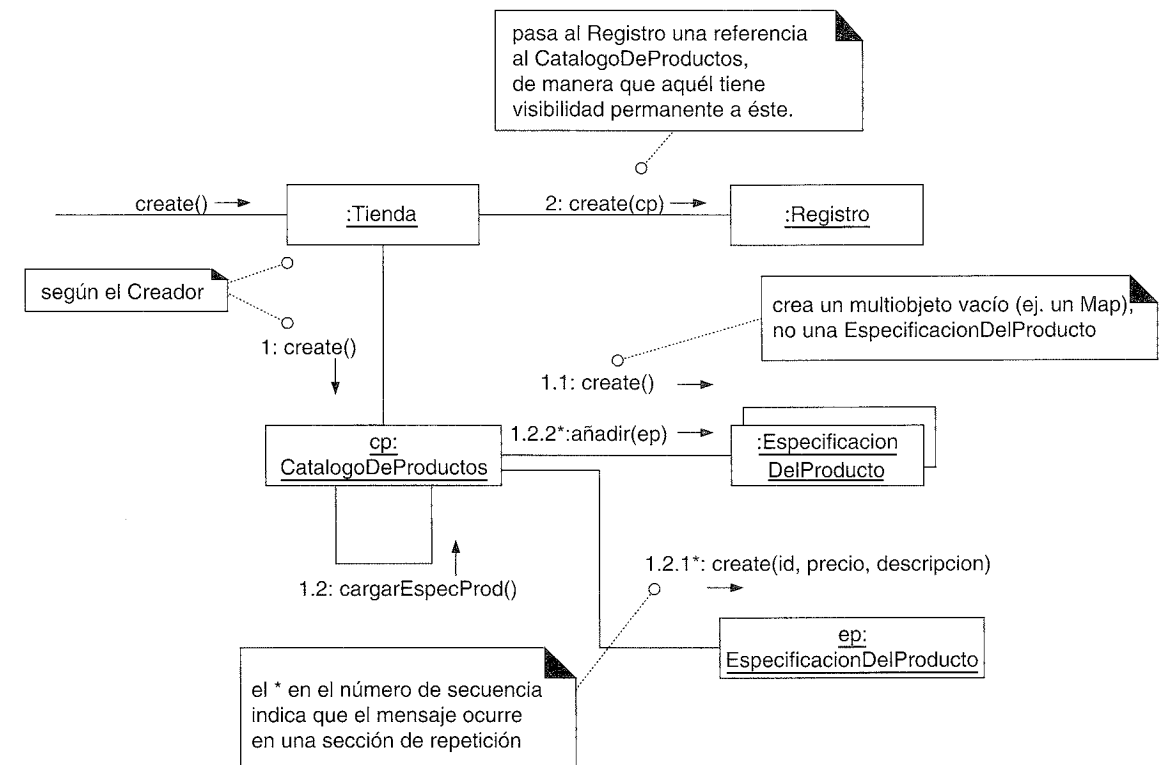


Figura 17.17. Creación del objeto del dominio inicial y los objetos siguientes.

Una desviación interesante entre el modelado del mundo real y el diseño se ilustra en el hecho de que el objeto software *Tienda* sólo crea un objeto *Registro*. Una tienda real podría albergar *muchos* registros o terminales de PDV reales. Sin embargo, estamos considerando un diseño software, no la vida real. En nuestros requisitos actuales, nuestra *Tienda* software sólo necesita crear una única instancia de un *Registro* software.

La multiplicidad entre las clases de objetos del Modelo del Dominio y el Modelo del Diseño podría no ser la misma.

17.9. Conexión de la capa de UI con la capa del dominio

Como se ha discutido brevemente, las aplicaciones se organizan en capas lógicas que separan los aspectos más importantes de la aplicación, como la capa de UI (para las cuestiones de la UI) y una capa de “dominio” (para las cuestiones de la lógica del dominio).

Entre los diseños típicos según los cuales los objetos de la capa del dominio son visibles a los objetos de la capa de la UI encontramos los siguientes:

- Una rutina de inicialización (por ejemplo, un método *main* en Java) crea tanto un objeto de la UI como un objeto del dominio, y pasa el objeto del dominio a la UI.

- Un objeto de la UI recupera el objeto del dominio de una fuente bien conocida, como un objeto factoría que es responsable de la creación de los objetos del dominio.

La muestra de código que se presentó anteriormente es un ejemplo del primer enfoque:

```
public class Main
{
    public static void main (String[] args)
    {
        Tienda tienda = new Tienda();
        Registro registro = tienda.getRegistro();
        JFrameProcesarVenta frame = new JFrameProcesarVenta(registro);
        ...
    }
}
```

Una vez que el objeto de la UI está conectado a la instancia del *Registro* (el controlador de fachada en este diseño), puede reenviarle mensajes de eventos del sistema, como los mensajes *introducirArticulo* y *finalizarVenta* (ver Figura 17.18).

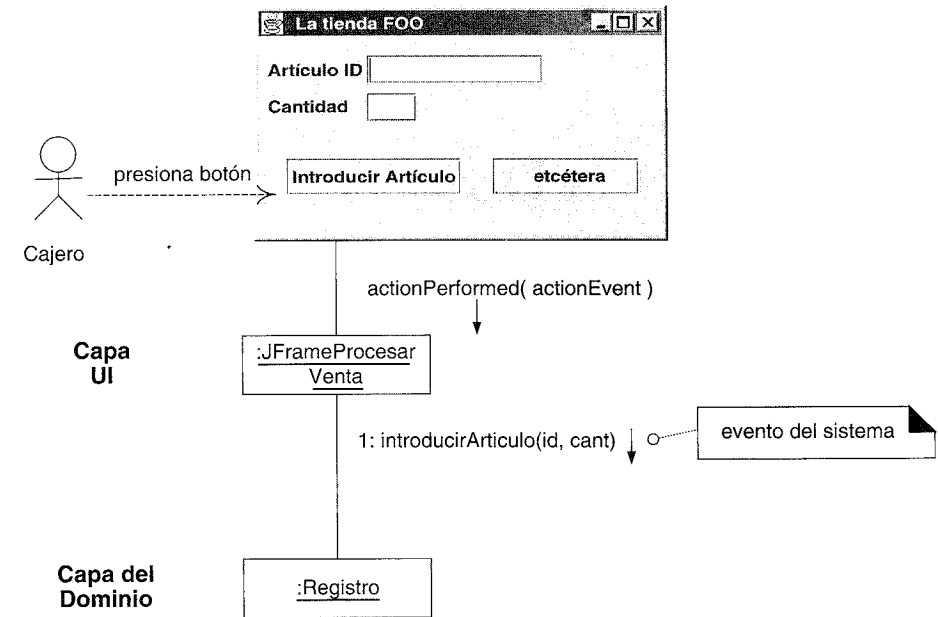


Figura 17.18. Conexión de la capas de UI y del dominio.

En el caso del mensaje *introducirArticulo*, la ventana necesita mostrar la suma parcial después de cada entrada. Hay varias soluciones de diseño:

- Añadir el método *getTotal* al *Registro*. La UI envía el mensaje *getTotal* al *Registro*, quien lo reenvía a la *Venta*. Esto tiene la posible ventaja de que mantiene bajo acoplamiento entre la UI y el modelo del dominio —la UI sólo conoce al objeto *Re-*

gistro—. Pero comienza a expandir el interfaz del objeto *Registro*, haciéndolo menos cohesivo.

- Una UI solicita una referencia al objeto *Venta* actual, y entonces cuando necesita el total (o cualquier otra información relacionada con la venta), envía el mensaje directamente a la *Venta*. Este diseño incrementa el acoplamiento entre la UI y el modelo del dominio. Sin embargo, como se estudió en la discusión del patrón GRASP Bajo Acoplamiento, un mayor acoplamiento por sí mismo no es un problema; más bien, en especial constituye un problema el acoplamiento entre cosas inestables. Asuma que decidimos que la *Venta* es un objeto estable que será una parte integral del diseño —lo que es muy razonable—. Entonces, el acoplamiento con la *Venta* no es un problema.

Como se ilustra en la Figura 17.19, este diseño sigue el segundo enfoque.

Nótese que en estos diagramas la ventana Java (*JFrameProcesarVenta*), que forma parte de la capa de UI, no es responsable de manejar la lógica de la aplicación. La ventana remite las solicitudes de trabajo (las operaciones del sistema) a la capa del dominio, por medio del *Registro*. Esto nos lleva al siguiente principio de diseño:

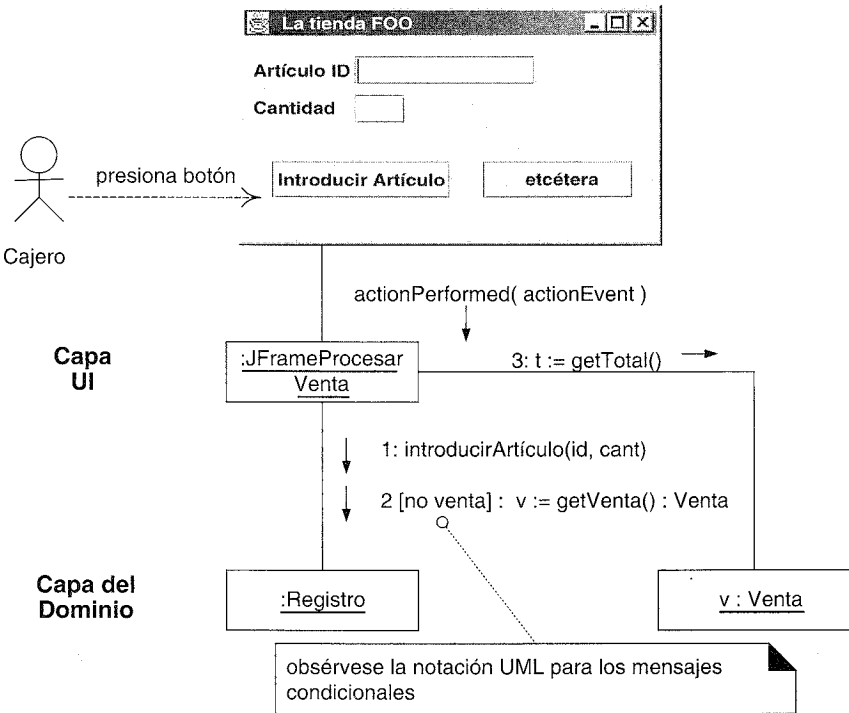


Figura 17.19. Conexión de las capas de la UI y del dominio.

Responsabilidades de la capa del dominio y de interfaz

La capa de UI no debería tener ninguna responsabilidad de la lógica del dominio. Sólo debería ser responsable de las tareas de la interfaz de usuario, como actualizar los elementos gráficos. La capa de UI debería remitir las solicitudes de las tareas orientadas al dominio a la capa del dominio, que es la responsable de manejarlas.

17.10. Realizaciones de casos de uso en el UP

Las realizaciones de los casos de uso forman parte del Modelo de Diseño del UP. Este capítulo ha resaltado la elaboración de los diagramas de interacción, pero es común y se recomienda que se elaboren los diagramas de clases en paralelo. Los diagramas de clases se estudiarán en el Capítulo 19.

Tabla 17.1. Muestra de los artefactos UP y evolución temporal. c – comenzar; r – refinar.

Disciplina	Artefacto Iteración →	Inicio I1	Elab. E1...En	Const. C1...Cn	Trans. T1...T2
Modelado del Negocio	Modelo del Dominio		c		
Requisitos	Modelo de Casos de Uso (DSS)	c	r		
	Visión	c	r		
	Especificación complementaria	c	r		
	Glosario	c	r		
Diseño	Modelo de Diseño		c	r	
	Documento de Arquitectura SW		c	r	
	Modelo de Datos		c	r	
Implementación	Modelo de Implementación		c	r	r
Gestión del Proyecto	Plan de Desarrollo SW	c	r	r	r
Pruebas	Modelo de Pruebas		c	r	
Entorno	Marco de Desarrollo	c	r		

Fases

Inicio: El Modelo de Diseño y las realizaciones de los casos de uso normalmente no comenzarán hasta la elaboración porque comprende decisiones de diseño detalladas que son prematuras durante la fase de inicio.

Elaboración: Durante esta fase, podrían crearse las realizaciones de los casos de uso para los escenarios más significativos desde el punto de vista de la arquitectura o de más riesgo del diseño. Sin embargo, no se harán los diagramas de UML para cada escenario, y no necesariamente con todo detalle o de grano fino. La idea es realizar los diagramas de interacción para las realizaciones de los casos de uso claves que se benefician de algún estudio anticipado y exploración de alternativas, centrándose en las decisiones de diseño más importantes.

Construcción: Se crean las realizaciones de los casos de uso para el resto de problemas de diseño.

En el UP, el trabajo de la realización de los casos de uso es una actividad de diseño. La Figura 17.21 ofrece sugerencias sobre el momento y el lugar para llevar a cabo este trabajo.

17.11. Resumen

Lo esencial de un diseño de objetos lo constituyen el diseño de las interacciones de objetos y la asignación de responsabilidades. Las decisiones que se tomen pueden influir

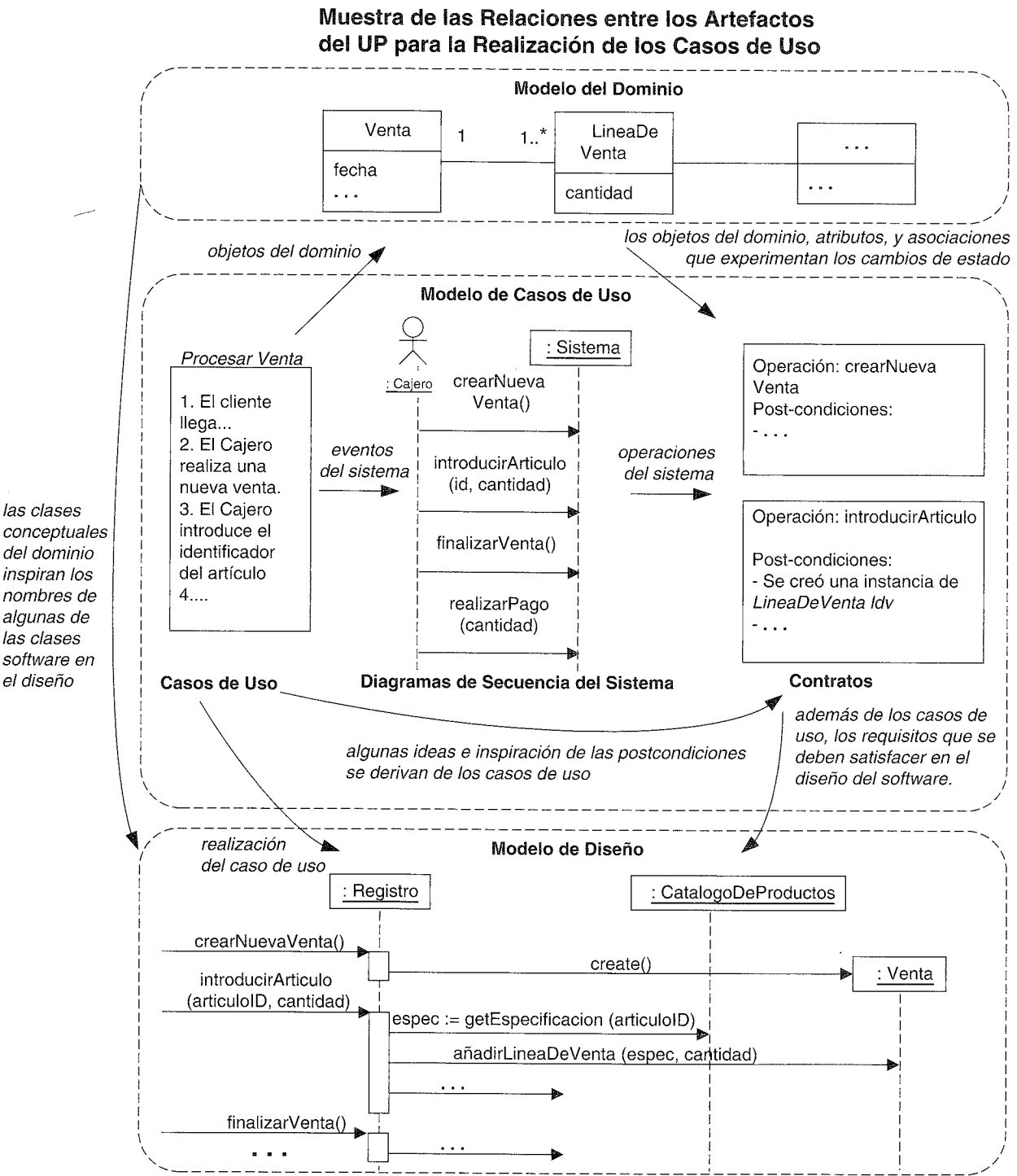


Figura 17.20. Muestra de la influencia entre los artefactos UP.

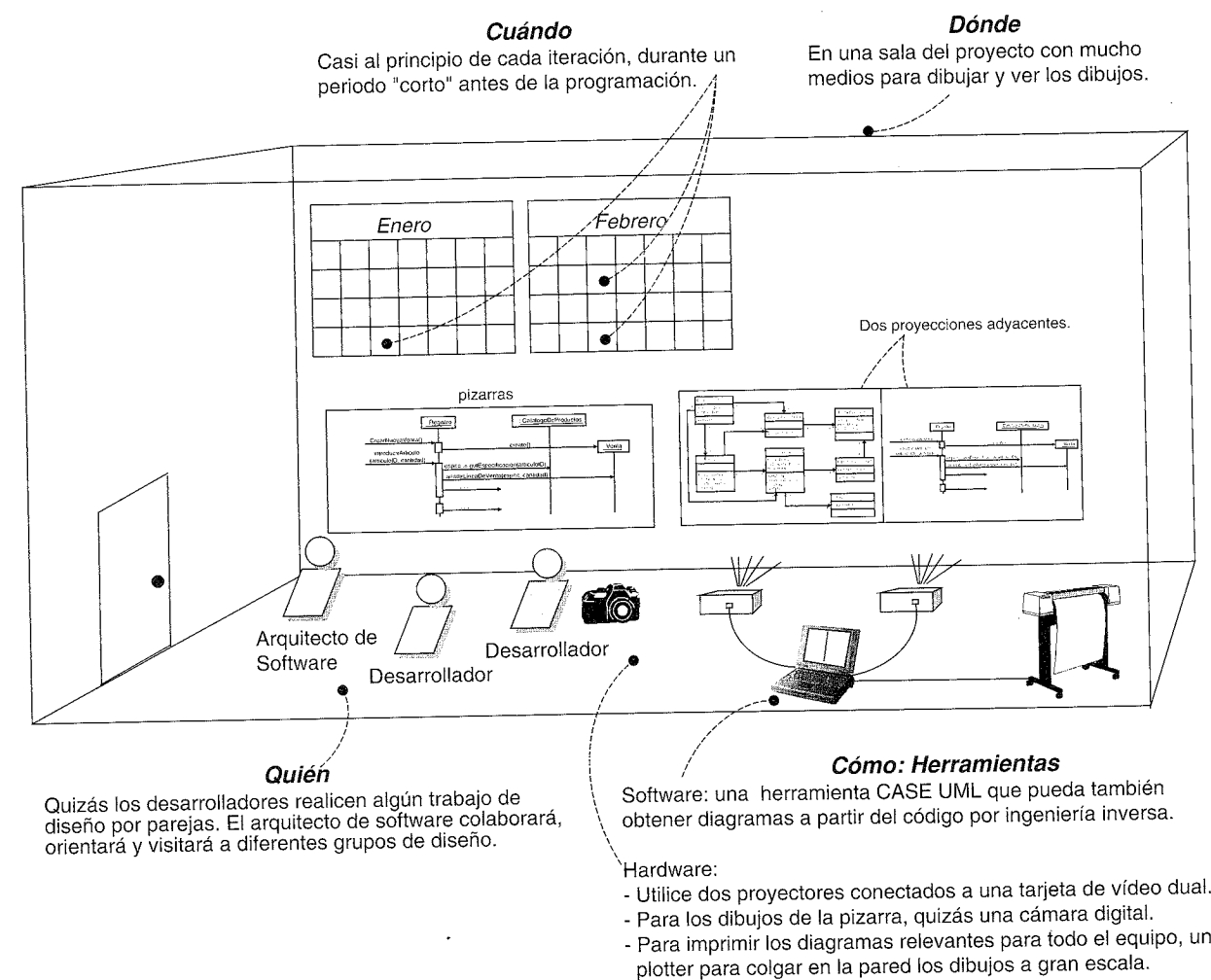
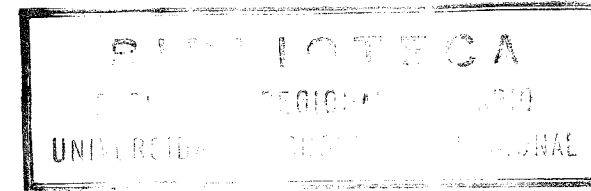


Figura 17.21. Proceso y establecimiento del contexto.

profundamente en la extensibilidad, claridad y mantenimiento del sistema software de objetos, además de en el grado y calidad de los componentes reutilizables. Existen principios que se pueden seguir para tomar las decisiones en cuanto a la asignación de responsabilidades; los patrones GRASP resumen algunos que los diseñadores orientados a objetos utilizan con frecuencia y están generalizados.



MODELO DE DISEÑO: DETERMINACIÓN DE LA VISIBILIDAD

Un matemático es una máquina que transforma café en teoremas.

Paul Erdős

Objetivos

- Identificar cuatro tipos de visibilidad.
- Diseñar para establecer la visibilidad.
- Ilustrar los tipos de visibilidad en la notación UML.

Introducción

La visibilidad es la capacidad de un objeto de ver o tener una referencia a otro. Este capítulo estudia las cuestiones de diseño relacionadas con la visibilidad.

18.1. Visibilidad entre objetos

Los diseños creados para los eventos del sistema (*introducirArticulo*, etcétera) ilustran mensajes entre objetos. Para que un objeto emisor envíe un mensaje a un objeto receptor, el receptor debe ser visible al emisor —éste debe tener algún tipo de referencia o apunador al objeto receptor—.

Por ejemplo, el mensaje *getEspecificacion* enviado desde un *Registro* a un *CatalogoDeProductos* implica que la instancia de *CatalogoDeProductos* es visible a la instancia de *Registro*, como se muestra en la Figura 18.1.

Cuando creamos un diseño de objetos que interaccionan, es necesario asegurar que se presenta la visibilidad adecuada para soportar la interacción de mensajes.

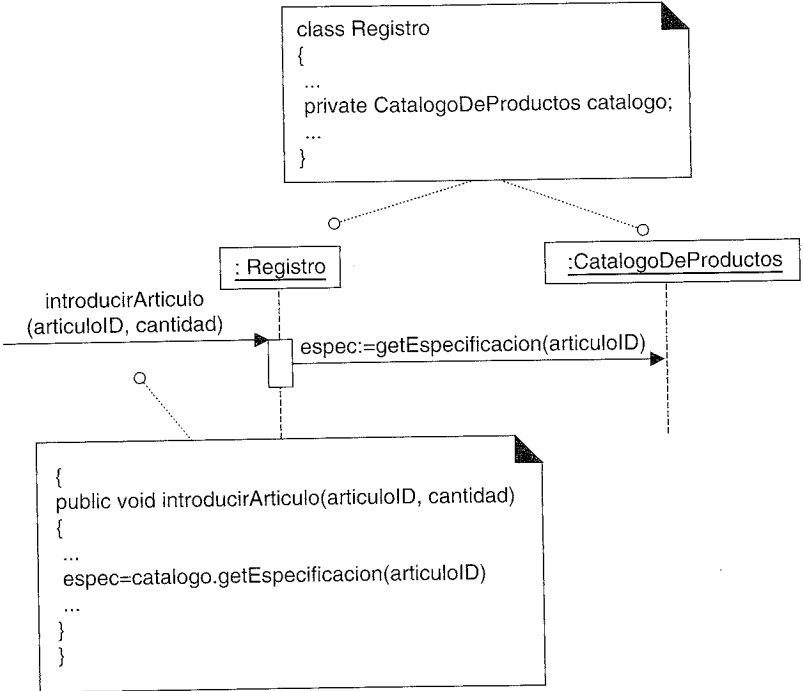


Figura 18.1. Se requiere visibilidad desde el Registro al CatalogoDeProductos¹.

UML tiene una notación especial para representar la visibilidad; este capítulo presenta varios tipos de visibilidad y su descripción.

18.2. Visibilidad

En su uso habitual, por **visibilidad** se entiende la capacidad de un objeto de “ver” o tener una referencia a otro objeto. De manera más general, está relacionado con el tema del alcance: ¿se encuentra un recurso (tal como una instancia) al alcance de otro? Hay cuatro formas comunes de alcanzar la visibilidad desde un objeto A a un objeto B:

- **Visibilidad de atributo:** B es un atributo de A.
- **Visibilidad de parámetro:** B es un parámetro de un método de A.
- **Visibilidad local:** B es un objeto local (no un parámetro) en un método de A.
- **Visibilidad global:** B es de algún modo visible globalmente.

El motivo para tener en cuenta la visibilidad es la siguiente:

Para que un objeto A envíe un mensaje a un objeto B, B debe ser visible a A.

Por ejemplo, para crear un diagrama de interacción en el que se envía un mensaje desde una instancia de *Registro* a una instancia de *CatalogoDeProductos*, el *Registro*

¹ En éste y en los siguientes ejemplos de código, podrían hacerse simplificaciones del lenguaje en aras de la brevedad y claridad.

debe tener visibilidad del *CatalogoDeProductos*. Una solución típica para la visibilidad es mantener una referencia a una instancia del *CatalogoDeProductos* como atributo del *Registro*.

Visibilidad de atributo

La **visibilidad de atributo** desde A a B existe cuando B es un atributo de A. Es una visibilidad relativamente permanente porque persiste mientras existan A y B. Ésta es una forma de visibilidad muy común en los sistemas orientados a objetos.

Para ilustrarlo, sea una definición de clase en Java para el *Registro*, una instancia de *Registro* tendría visibilidad de atributo a un *CatalogoDeProducto*, puesto que es un atributo (variable de instancia Java) del *Registro*.

```
public class Registro
{
  ...
  private CatalogoDeProductos catalogo;
  ...
}
```

Se requiere esta visibilidad porque en el diagrama de *introducirArticulo* que se muestra en la Figura 18.2, un *Registro* necesita enviar el mensaje *getEspecificacion* a un *CatalogoDeProductos*:

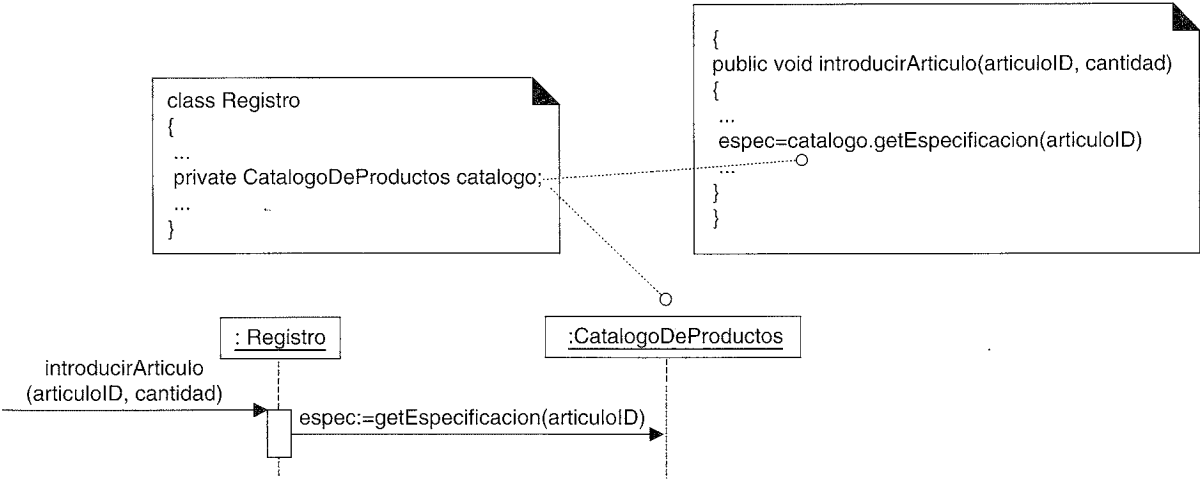


Figura 18.2. Visibilidad de atributo.

Visibilidad de parámetro

La **visibilidad de parámetro** desde A a B existe cuando B se pasa como parámetro a un método de A. Es una visibilidad relativamente temporal porque persiste sólo en el alcance del método. Después de la visibilidad de atributo, es la segunda forma más común de visibilidad en los sistemas orientados a objetos.

Para ilustrarlo, cuando se envía el mensaje *crearLineaDeVenta* a la instancia de *Venta*, se pasa como parámetro una instancia de *EspecificacionDelProducto*. En el alcance del método *crearLineaDeVenta*, la *Venta* tiene visibilidad de parámetro de una *EspecificacionDelProducto* (ver Figura 18.3).

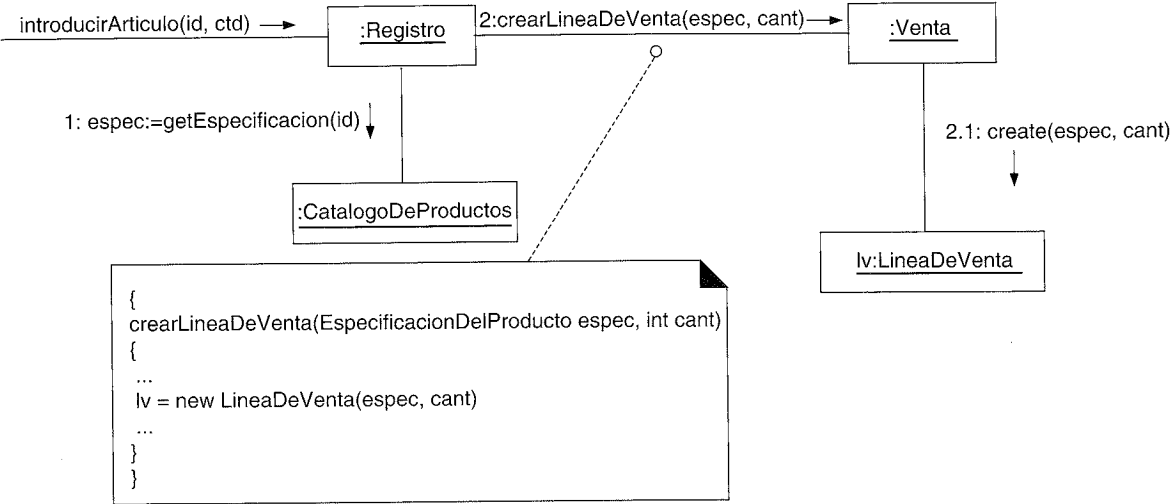


Figura 18.3. Visibilidad de parámetro.

Es habitual transformar la visibilidad de parámetro en visibilidad de atributo. Por ejemplo, cuando la *Venta* crea una nueva *LineaDeVenta*, le pasa en el método de inicialización una *EspecificacionDelProducto* (sería su **constructor** en Java y C++). En el método de inicialización, se asigna el parámetro a un atributo; por tanto, se establece visibilidad de atributo (ver Figura 18.4).

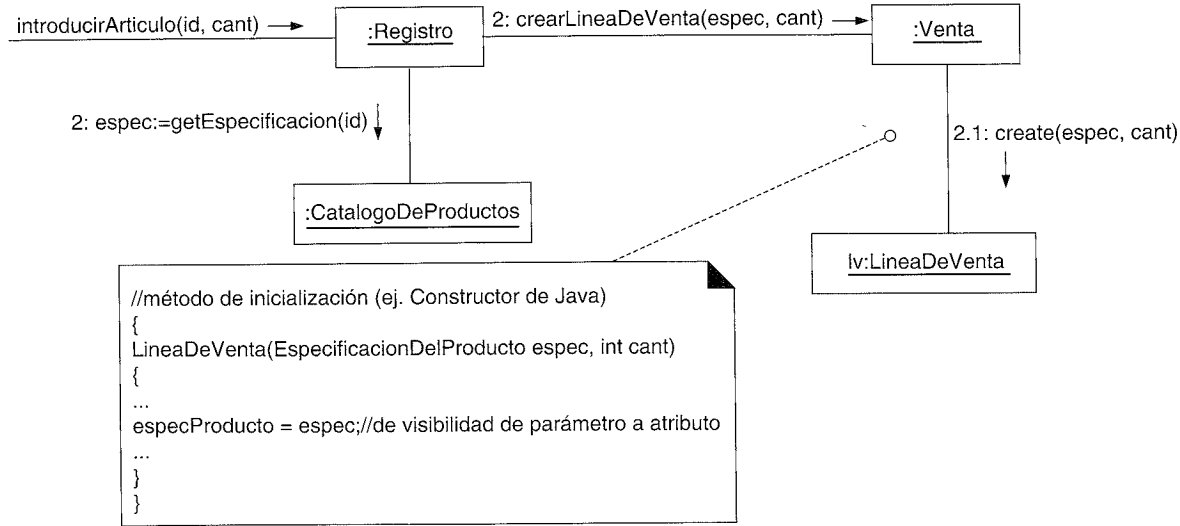


Figura 18.4. De visibilidad de parámetro a visibilidad de atributo.

Visibilidad local

La **visibilidad local** desde A a B existe cuando B se declara como un objeto local en un método de A. Es una visibilidad relativamente temporal porque sólo persiste en el alcance del método. Después de la visibilidad de parámetro, es la tercera forma de visibilidad más común en los sistemas orientados a objetos.

Dos medios comunes de alcanzar la visibilidad local son:

- Crear una nueva instancia local y asignarla a una variable local.
- Asignar a una variable local el objeto de retorno de la invocación a un método.

Como con la visibilidad de parámetro, es habitual transformar la visibilidad declarada localmente en visibilidad de atributo.

Un ejemplo de la segunda variación (asignación a una variable local del objeto de retorno) se puede encontrar en el método *introducirArticulo* de la clase *Registro* (Figura 18.5).

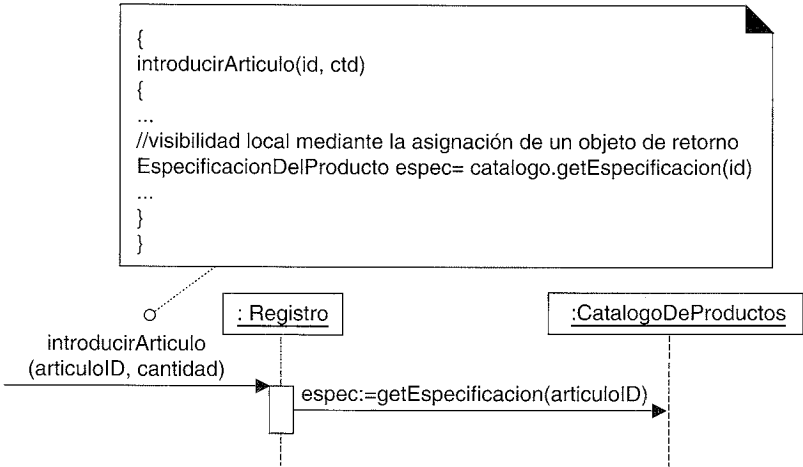


Figura 18.5. Visibilidad local.

Una versión sutil de la segunda variación es cuando el método no declara explícitamente una variable, sino que implícitamente existe una como resultado de un objeto de retorno de una invocación a un método. Por ejemplo:

```
//hay una visibilidad local implícita al objeto foo
//resultado de la llamada a getFoo
unObjeto.getFoo().hacerBar();
```

Visibilidad global

La **visibilidad global** de A a B existe cuando B es global a A. Es una visibilidad relativamente permanente porque persiste mientras existan A y B. Es la forma menos común de visibilidad en los sistemas orientados a objetos.

Un medio de conseguir visibilidad global es asignar una instancia a una variable global, lo que es posible en algunos lenguajes, como C++, pero no en otros, como Java.

El método que se prefiere para conseguir la visibilidad global es utilizar el patrón Singleton [GHJV95], que se presentará en un capítulo posterior.

18.3. Representación de la visibilidad en UML

UML incluye notación para representar el tipo de visibilidad en un diagrama de colaboración (ver Figura 18.6). Estos adornos son opcionales y normalmente no se exigen; son útiles cuando se necesita alguna aclaración.

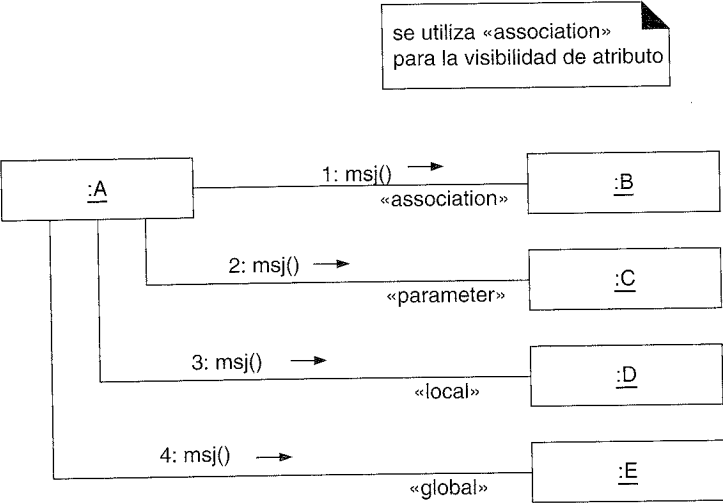


Figura 18.6. Implementación de los estereotipos para la visibilidad.

MODELO DE DISEÑO: CREACIÓN DE LOS DIAGRAMAS DE CLASES DE DISEÑO

Iterar es humano, ser recursico, divino.

Anónimo

Objetivos

- Crear los diagramas de clases de diseño (DCD).
- Identificar las clases, métodos y asociaciones que se van a mostrar en un DCD.

Introducción

Una vez terminados los diagramas de interacción para las realizaciones de los casos de uso de la iteración actual de la aplicación de PDV NuevaEra, es posible identificar la especificación de las clases software (e interfaces) que participan en la solución software, y añadirles detalles de diseño, como los métodos.

UML proporciona la notación para representar los detalles de diseño en los diagramas de clase; en este capítulo, vamos a estudiarla y a crear los DCD.

19.1. Cuándo crear los DCD

Aunque esta presentación de los DCD viene después de la creación de los diagramas de interacción, en la práctica normalmente se crean en paralelo. Al comienzo del diseño se podrían esbozar muchas clases, nombres de métodos y relaciones aplicando los patrones para asignar responsabilidades, antes de la elaboración de los diagramas de interacción. Es posible y deseable elaborar algo de los diagramas de interacción, actualizar entonces los DCD, después extender los diagramas de interacción algo más, y así sucesivamente.

Estos diagramas de clases podrían utilizarse como una notación más gráfica alternativa a las tarjetas CRC para recoger las responsabilidades y colaboraciones.

19.2. Ejemplo de DCD

El DCD de la Figura 19.1 ilustra una definición software parcial de las clases *Registro* y *Venta*.

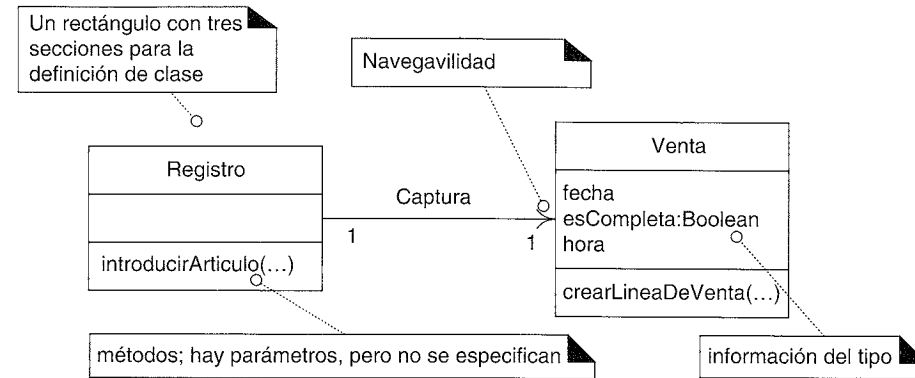


Figura 19.1. Ejemplo de diagrama de clases de diseño.

Además de las asociaciones y atributos básicos, el diagrama se amplía para representar, por ejemplo, los métodos de cada clase, información del tipo de los atributos, visibilidad de los atributos y navegación entre los objetos.

19.3. Terminología del DCD y el UP

Un **diagrama de clases de diseño** (DCD) representa las especificaciones de las clases e interfaces software (por ejemplo, las interfaces de Java) en una aplicación. Entre la información general encontramos:

- Clases, asociaciones y atributos.
- Interfaces, con sus operaciones y constantes.
- Métodos.
- Información acerca del tipo de los atributos.
- Navegabilidad.
- Dependencias.

A diferencia de las clases conceptuales del Modelo del Dominio, las clases de diseño de los DCD muestran las definiciones de las clases software en lugar de los conceptos del mundo real.

El UP no define de manera específica ningún artefacto denominado “diagrama de clases de diseño”. El UP define el Modelo de Diseño, que contiene varios tipos de diagramas, que incluye los diagramas de interacción, de paquetes, y los de clases. Los diagramas de clases del Modelo de Diseño del UP contienen “clases de diseño” en términos del UP. De ahí que sea común hablar de “diagramas de clases de diseño”, que es más corto que, e implica, “diagramas de clases en el Modelo de Diseño”.

19.4. Clases del Modelo de Dominio vs. clases del Modelo de Diseño

Seamos reiterativos, en el Modelo de Dominio del UP, una *Venta* no representa una definición software, sino que se trata de una abstracción de un concepto del mundo real acerca del cual estamos interesados en realizar una declaración. En cambio, los DCD expresan —para la aplicación software— la definición de las clases como componentes software. En estos diagramas, una *Venta* representa una clase software (ver Figura 19.2).

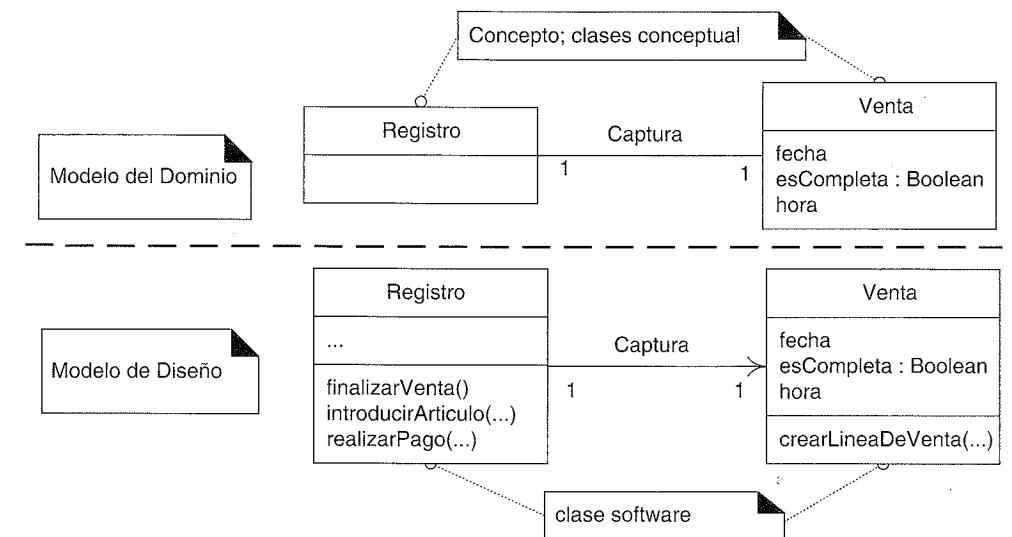


Figura 19.2. Clases del Modelo del Dominio vs. clases del Modelo de Diseño.

19.5. Creación de un DCD del PDV NuevaEra

Identificación y representación de las clases software

El primer paso en la creación de los DCD como parte del modelo de la solución es identificar aquellas clases que participan en la solución software. Se pueden encontrar examinando todos los diagramas de interacción y listando las clases que se mencionan.

Para la aplicación del PDV son:

Registro	Venta
CatalogoDeProductos	EspecificacionDelProducto
Tienda	LineaDeVenta
Pago	

El siguiente paso es dibujar un diagrama de clases para estas clases e incluir los atributos que se identificaron previamente en el Modelo del Dominio que también se utilizan en el diseño (ver Figura 19.3).